Compiler Ch1–3｜考場列印手冊

> 黑白 A4 完整版 · 2026–10–14 · 八份註解程式的選檔 / 改題指南＋Ch1—3 三章完整速查表。

程式直接帶原始 .cpp；紙本用來找題型、看推導、核對輸入輸出及修改位置。本冊保留八題的完整題意、 基本測例、手動追蹤、40 個修改範例的改前 / 改後與執行結果，不需展開才能閱讀。

A｜先選題，再開檔

題型	Code 檔案（相對 Complier/Code/）	PDF 頁
HTML 標籤配對	`exam/short/html_check.cpp`	8
遞迴下降運算式	`exam/short/expression.cpp`	13
Scanner：沿用現有短程式	`exam/lexer.cpp`	17
NFA 直接判定字串	`exam/short/nfa_check.cpp`	22
HW5 固定四狀態 DFA	`exam/short/hw5_check.cpp`	28
ac 邊解析邊產 dc	`exam/short/ac_dc.cpp`	32
Stack VM：沿用現有短程式	`exam/stack_vm.cpp`	38
LL(1)：較低優先補充	`exam/ll1.cpp`	43

快速入口：版本 / 模式 p.3；Linux / I/O p.4；除錯 p.5；小考英文短答 p.6

相同檔名也可能有不同版本：exam/short/expression.cpp 是短版求值； exam/expression.cpp 是完整版本，預設 postfix。以本表完整相對路徑選檔。

如何使用這本手冊

1. 圈出輸出要求：合法判定、Token、Tree、AST、Postfix、DFA 表、目標碼，或執行結果。
2. 先看該題輸入規格與範圍，再用配套 .in / .expected 跑一次原版。
3. 按原檔行號找到修改區塊；每個變體獨立套用，多項合併後重新找位置。
4. 以題目樣例確認輸出，再加空字串、邊界、非法字元、巢狀與長輸入等測例。
5. 需要手算時查章節附錄；教材頁碼、教材解答與補充標示照原版保留。

程式長行在紙本僅作視覺折行，修改時以 .cpp 原檔對照。PDF 最底端 Print handbook N / TOTAL 是本冊連續頁碼；附錄原有 01 / 06 等是各章自身編號。 此 HTML 僅含前半部程式指南，三章完整內容已合併於 PDF。

章節內容索引：三章完整保留

現有教材伴讀速查表全部納入；來源 PPT / 教材頁碼與補充辨析保留在各頁。 HTML parser 的歷屆線索未附原卷，因此程式題標明模擬規格。 FIRST / FOLLOW / LL(1) 保留為較低優先延伸，正式範圍尚未確認。

原章節頁	內容	本冊 PDF 頁
Ch1・原第 1 頁	編譯器定義與目標碼	48
Ch1・原第 2 頁	Compiler / Interpreter 與編譯流程	49
Ch1・原第 3 頁	Scanner C 實作與前後端	50
Ch1・原第 4 頁	Bootstrapping 與 T 圖	51
Ch1・原第 5 頁	Optimizer：矩陣乘法	52
Ch1・原第 6 頁	HW 4 / 8 / 11 與短答	53
Ch1・原第 7 頁	名詞速查與自測	54
Ch2・原第 1 頁	詞法、Token 與英文短答	55
Ch2・原第 2 頁	15 條 ac 文法、Predict 與 Parser	56
Ch2・原第 3 頁	推導、Parse Tree 與 AST	57
Ch2・原第 4 頁	Symbol Table、型別與錯誤	58
Ch2・原第 5 頁	dc 產碼與 Stack Trace	59
Ch2・原第 6 頁	HW 1 / 2 / 3 與 Scanner 擴充	60
Ch3・原第 1 頁	中英術語與階段速查	61
Ch3・原第 2 頁	六類題目解題步驟	62
Ch3・原第 3 頁	CFG 與概念英文短答	63
Ch3・原第 4 頁	AST 畫法與優先序	64
Ch3・原第 5 頁	RE、Identifier 與 Exponent	65
Ch3・原第 6 頁	DFA / NFA 與 Scanner 行為	66
Ch3・原第 7 頁	RE→NFA 與 (bc)*d	67
Ch3・原第 8 頁	NFA→DFA 子集構造	68
Ch3・原第 9 頁	(a|b)*abb 構造與追蹤	69
Ch3・原第 10 頁	HW5(a) 完整表與短答	70

教材已有疑義：Ch2 p.34 未附引用圖、p.43 解答與型別規則的差異、 Scanner 小數位數及科學記號規則的差異，均沿用既有辨析。不要把補充版本當成教材逐字原解。

同一題型，輸出要求不同就換版本

以下路徑相對 Complier/Code/。短版能辨識字串，不代表它也會畫樹或列 DFA 表。 評測若不傳命令列參數，將 main 的預設模式改成題目需要的模式。

題目要求	原檔路徑	模式 / 關鍵設定
HTML 只判斷合法	`exam/short/html_check.cpp`	無參數；VALID / INVALID
HTML Token / Tree / Text	`exam/html_parser.cpp`	--tokens / --tree / --text；預設 --check
每行算式求值 / Postfix / AST	`exam/short/expression.cpp`	MODE：eval / postfix / ast
深巢狀 / 迭代式算式 Parser	`exam/expression.cpp`	--postfix（預設） / --ast / --eval
C-like Token、字串、註解	`exam/lexer.cpp`	keywords / ops 清單可改
教材 ac Scanner / Parse Tree / AST	`ch02/ac_homework.cpp`	--tokens / --parse-tree / --ast / --typed-ast
ac 型別檢查 / 產 dc	`exam/short/ac_dc.cpp`	只產 dc，不直接印數值
ac 完整型別 / 摺疊 / 產碼	`ch02/ac_homework.cpp`	--check / --dc（預設）；可加 --fold
給 NFA，僅判字串	`exam/short/nfa_check.cpp`	無參數；# 是 epsilon
給 NFA，產 DFA 表 / 集合追蹤	`exam/nfa_to_dfa.cpp`	--table / --trace；預設 --check
固定 HW5(a) / (d) 字串辨識	`exam/short/hw5_check.cpp`	HW5A=true / false
固定 HW5 原 NFA→subset DFA	`ch03/automata_homework.cpp`	5a --table / 5d --table
Stack 指令執行	`exam/stack_vm.cpp`	PRINT 會 pop；不是 dc
通用 FIRST / FOLLOW / LL(1)	`exam/ll1.cpp`	預設表格；--check 判 Token 序列

HTML parser 先判斷題意

- **合法判定：**START 入 stack，END 配 top，EOF 為空；數量相同不保證巢狀正確。
- **列 Token：**分辨 START / END / TEXT / COMMENT；是否檢查配對看題目要求。
- **輸出樹：**stack 的 top 是新節點的 parent，要保留 children 與文字次序。
- **抽文字：**是否保留空白、跳過 script / style、解碼 entity、替 br 換行？現有 --text 串接來源 TEXT，未做這些轉換。
- **邊界：**引號內 > 不切 Tag；註解另找 -->；void 不入 stack。 本模擬子集以 <x/> 自閉合，不等於完整瀏覽器的 HTML 規則。

> 程式的 I/O 與支援範圍就是使用條件。mock 題型可用來改題，實際輸入輸出仍按考卷。

讀題與跨平台輸入輸出：逐項核對

核對項目	先圈題目敘述	本程式的易錯處
案例界線	首行 T / 每行一案 / 整份 EOF 一案 / sentinel	跨行 HTML、ac 不可誤用每行一案。
空白與空行	空行是 epsilon 案例，還是忽略？空白是內容，還是分隔？	NFA / HW5 空行要測；expression 空白行目前是錯誤。
State 編號	0..n−1，還是 1..n？	1 起算要同步改 valid 與容器大小。
大小寫	Tag、Keyword、Instruction 是否區分大小寫？	只改允許的字元；literal 不可整份轉小寫。
數字	整數除法 / 浮點；小數位數 / 有效位數；e / E	fixed+setprecision(2) 是兩位小數；不加 fixed 是有效位數。
輸出格式	YES / NO、Case 編號、分隔空白、EOF Token	不印提示字、debug、額外的樹或表。
錯誤案例	印指定文字、拒絕單案，還是終止全部？	多份原檔暫存 stdout，任一案錯誤會取消整份輸出。
複雜度	長度、巢狀深度、State 上限、Time Limit	短 expression 深度上限 256；subset DFA 可能指數膨脹。

Linux / DOMjudge 編譯與執行

```
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror html_check.cpp -o main
./main < input.txt > actual.txt
diff -u expected.txt actual.txt
```

每份都有自己的 main()，選一份編譯，不要把所有 .cpp 同時連結。 使用標準 C++17；不加入 windows.h、conio.h、getch、gets、system("pause") 或 system("cls")。

getline 與 CRLF

getline 吃掉 LF，但最後可能留下 CR。 以下為片段，未單獨編譯；每次讀到一行後去掉**末尾** CR，不刪內容中所有空白。

```
while (getline(cin, line)) {
    if (!line.empty() && line.back() == '\r') line.pop_back();
    // 處理一個案例；是否略過空行依題意。
}
```

混用 cin >> T 和 getline 時，先讀完 T 所在行的剩餘內容，再讀第一案。 若空行代表合法案例，不用 cin >> ws，它會吞掉空行。

模式、輸出與原碼行號

行號對應 2026-10-14 含中文註解的原檔，八份共 806 行。 每個變體從原版獨立套用；插入程式後後面行號會位移，可搜尋「改前」字串重新定位。 範例裡 stdout：空 / stderr / exit 1 是手冊說明，不是要印在 stdout 的文字。

出錯時依症狀找位置

現象	先檢查	最小定位方式
編譯錯誤	先看第一個診斷：include、括號、引號、型別、選檔。	原檔通過後，一次改一個區塊；使用 C++17。
輸出多了 / 少了行	檢查 T 與 EOF 是否混用、空行是否被跳過。	stdout 只留指定內容；Case 編號按題意起算。
小數答案不同	有效位數、fixed、小數位數與整數除法。	ac 精確十進位字串不可先轉 double 再印。
配對 / 巢狀失敗	stack top、引號、void 與 raw text。	測交錯關閉與含 > 的屬性。
運算順序不同	優先序、結合性與 stack pop 順序。	100−30−1=69；8/2/2=2；先 pop 右 b 再左 a。
NFA 判定不同	初始 closure；move 後再 closure；讀完才看 final。	測空字串、epsilon cycle、final 有外出邊。
Runtime Error	空 stack / 越界 / 未定義變數 / 零除 / 深遞迴。	先測最小合法輸入，再測允許邊界。
Time Limit	迴圈是否推進游標，epsilon 是否重複進 todo。	AST 字串拼接可能 $O(n^2)$；深 / 長算式換完整迭代版。

提交前的測例清單

1. 全部題目樣例，逐 byte 比對輸出；空白、最後換行與 Case 字樣都要對。
2. 最短合法例、空輸入 / 空案例，以及同類的最短非法例。
3. 一層與多層巢狀；不平衡、交錯關閉；引號與 escaped quote。
4. 最高優先序、同級左結合、負 operand、零值與整數 / 浮點混合。
5. 多案連續輸入；每案游標、stack、輸出與深度等狀態要重設。
6. LF 與 CRLF 各測一次；只有空白的尾端也按題目規格處理。

> 八份原檔已通過 610 項 native 檢查與 sanitizer；40 個修改範例另有 82 次樣例檢查。 這次只編製文件，核對來源與報告的 SHA-256，沒有修改程式，也未新增 DOMjudge Accepted 證據。

小考原題與可直接作答的英文

第一次五題原文由使用者提供；英文答案是既有筆記依教材整理的參考答案，非教師公布的逐字解答。第二次小考只提供題意，AST 的原程式未附。

Quiz 1．1｜Compiler 與 Interpreter

原題：What is the difference between compiler and interpreter ?

> A compiler translates a source program into a target program, which is then executed. An interpreter executes a source program or its intermediate representation using input data and directly produces results. The main distinction is translation into a target program versus execution of the program.

Compiler 產出目標程式；Interpreter 執行程式而產出結果。不要用「一定逐行」或「完全不翻譯」當定義。

Quiz 1．2｜Token 是什麼

原題：Explain what a token is ?

> A token is a basic lexical unit recognized by the scanner, such as a keyword, identifier, literal, operator, or punctuation symbol. A token has a kind and may carry an attribute, such as an identifier name or a literal value. The actual character sequence forming a token is called its lexeme.

Token kind 是類別，lexeme 是來源字串，attribute 是附帶資訊；int sum = 12; 有五個來源 tokens。

Quiz 1．3｜Windows x86 的 C 程式能否在 Linux x86 執行

原題：A program written in C can be compile and execute in Win (x86) , is it also work in Linux (x86) ?

> Not necessarily. A Windows x86 executable normally cannot run directly on Linux x86, because the executable format, operating-system interfaces, ABI, and runtime libraries may differ. The C source code can be recompiled and linked for Linux if it is portable and its dependencies are available. Windows-specific code must be ported first.

分清原始碼與二進位執行檔。相同 x86 不等於相同 OS、ABI 或函式庫；可攜原碼在 Linux 重新編譯。

Quiz 1．4｜取得可在 ARM 執行的 javac

原題：We have a java compiler written in C on X86, how can we get a javac for arm device ?

> Use a C cross-compiler that runs on x86 and generates code for the ARM device. Compile the C source of the Java compiler and link it with the appropriate ARM target libraries to produce an ARM executable of javac. Transfer it and its required runtime dependencies to the ARM device. Alternatively, build the same C source with a native C compiler on ARM.

用在 x86 執行、產生 ARM 程式的 C 交叉編譯器，編譯 javac 的 C 原碼。產出的 javac 在 ARM 跑，仍將 Java 翻成 bytecode；不是把 javac 的 bytecode 後端改成 ARM。

Quiz 1．5｜Grammatical error 與 Semantic error

原題：Explain grammatical error and Semantic error

> A grammatical error, also called a syntax error, violates the grammar of a language, such as a missing expression or an unmatched parenthesis. A semantic error violates the language's meaning rules even when the syntax is valid, such as using an undeclared identifier or calling a non-function object. The parser checks syntax, while the semantic analyzer checks static semantic constraints such as names and types.

Parser 檢查排列是否符合文法；語意分析檢查宣告、名稱與型別。a=; 是缺 operand；使用未宣告變數是語意錯誤。

Quiz 2．1｜Why use CFG?

原題：Why use CFG (context free grammer) instead of others ?

> A context-free grammar can describe recursive and nested syntax, such as balanced parentheses and nested statements, beyond the expressive power of regular expressions. Its productions also provide structure for parsing and AST construction. Semantic constraints, such as declarations and types, are checked separately.

補充參考答案：RE 適合 Token 模式；CFG 可描述遞迴與任意巢狀結構，又能交給 Parser 建樹。CFG 不能取代名稱 / 型別檢查。更多例子見 Ch3 附錄。

Quiz 2．2｜畫短程式的 AST

原題：wrtie down the AST for a short code

原程式未提供；以下另編練習：`a+b*c` → `(+ a (* b c))`；`100-30-1` → `(- (- 100 30) 1)`。

1. 辨認最外層低優先序的運算子，作為根。
2. 依優先序、括號與結合性切左右子樹。
3. AST 保留有意義的運算 / 宣告 / 敘述，不保留文法輔助節點與括號 token。
4. 若要求 decorated AST，查 symbol table，由葉往根標型別，必要時加 int2float。

01 HTML 標籤配對

題目內容

> 歷屆 HTML parser 是使用者提供的線索，未提供原考卷；以下是完整模擬題。

請撰寫程式讀入一份 HTML fragment，判斷所有非 void、非自閉合元素是否正確巢狀。開始標籤必須依相反順序關閉；END 名稱須與最內層未關閉元素相同。名稱不分 ASCII 大小寫。支援 boolean、單雙引號與無引號屬性、註解、短 DOCTYPE html；script/style/textarea/title 內容讀到匹配 END。多個根元素與空 fragment 均合法。

輸入規格

stdin 全部內容到 EOF 是一個案例，可能跨行。普通元素必須顯式關閉；本模擬題將 <x/> 視為自閉合。無額外案例數 T。

輸出規格

合法印 VALID，否則印 INVALID，每次只印一行；這兩種分類都正常結束。

限制與適用範圍

不是完整 browser DOM：不恢復錯誤、不省略 end tag、不解碼 entity、不檢查內容模型。這裡的「合法」指指定子集。

Code 檔案名稱：`html_check.cpp` （exam/short/html_check.cpp）

```
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror html_check.cpp -o main
./main < input.txt
```

題目範例與邊界測例

輸入

```
<div id="main"><p title="a>b">Hello <b>world</
b></p><br><img src=x><!--ok--></div>
```

輸出

```
VALID
```

邊界測例 01

輸入

```
（空輸入）
```

輸出

```
VALID
```

邊界測例 02

輸入

```
<P>x</p><br>
```

輸出

```
VALID
```

邊界測例 03

輸入

```
<a><b></a></b>
```

輸出

```
INVALID
```

邊界測例 04

輸入	輸出
`<p a="a>b">x</p>`	`VALID`

邊界測例 05

輸入	輸出
`<p a=x/>`	`INVALID`

邊界測例 06

輸入	輸出
`<p a=x />`	`VALID`

邊界測例 07

輸入	輸出
`<!-- unclosed`	`INVALID`

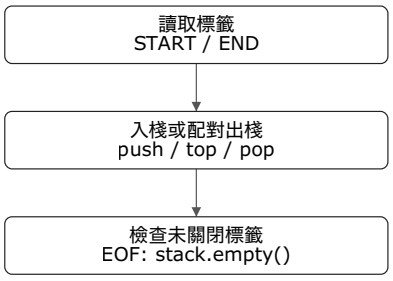
邊界測例 08

輸入	輸出
`<script>if(a<b){}</SCRIPT >`	`VALID`

邏輯與手動追蹤

START 入 stack；END 必須匹配頂端才 pop；EOF 時 stack 必須空。void 與自閉合標籤不入 stack；屬性引號內的 > 不結束標籤，註解與 raw text 另行跳過。

- <a>x：[] → [a] → [a,b] → [a] → []，接受。
- <a>：讀 時 top 是 b，立即拒絕。
- <p title="a>b">x ：引號中的 > 不切開 tag。
- 沒有關 p； 則已自閉合。



方框：處理步驟　箭頭：執行順序或呼叫關係

題目變動時：先查修改位置

行號以含註解的 .cpp 原檔為準。每個變體獨立套用；多項一起改時，重新定位，不要沿用位移後的行號。表格列修改索引，詳細改前 / 改後與測例在後文列出。

題目要求	原檔行號	完整改法
輸出 YES / NO	127	修改範例 01（後文）
一行一案	121–127	修改範例 02（後文）
XML 式：br 等也須配對關閉	32–33	修改範例 03（後文）
求最大巢狀深度	11；101；127	修改範例 04（後文）

題目要求	原檔行號	完整改法
首行 T，其後一行一案	121–127	修改範例 05（後文）

01 輸出 YES / NO：改前 / 改後與測例

將 "VALID\n" / "INVALID\n" 改成 "YES\n" / "NO\n"。

第 127 行：用改後取代此行 / 區塊

改前（原碼）

```
    cout << (valid ? "VALID\n" : "INVALID\n");
```

改後（對照修改）

```
    cout << (valid ? "YES\n" : "NO\n");
```

已試編並執行的變體測例：

輸入

```
<p/>
```

輸出

```
YES
```

02 一行一案：改前 / 改後與測例

以此區塊替換這段；跨行一案時保留原版： while (getline(cin, s)) { p = 0; bool valid = false; try { valid = check(); } catch (const runtime_error&) {} cout << (valid ? "VALID\n" : "INVALID\n"); } if (cin.bad()) return 1;

第 121–127 行：用改後取代此行 / 區塊

改前（原碼）

```
    s.assign(istreambuf_iterator<char>(cin),
{});
    // 讀取裝置失敗與文件語法非法不同：前者才用 exit
1。
    if (cin.bad()) { cerr << "input error\n";
return 1; }
    bool valid = false;
    // 語法錯誤也是分類結果 INVALID，所以這裡捕捉例
外，不把它當程式崩潰。
    try { valid = check(); } catch (const
runtime_error&) {}
    cout << (valid ? "VALID\n" : "INVALID\n");
```

改後（對照修改）

```
while (getline(cin, s)) {
    p = 0; bool valid = false;
    try { valid = check(); } catch (const
runtime_error&) {}
    cout << (valid ? "VALID\n" : "INVALID\n");
}
if (cin.bad()) return 1;
```

已試編並執行的變體測例：

輸入

```
<p/>
<p>
```

輸出

```
VALID
INVALID
```

03 XML 式：br 等也須配對關閉：改前 / 改後與測例

將兩行名單改成 const set<string> empty{}; ；
 仍自閉合。

第 32—33 行：用改後取代此行 / 區塊

改前（原碼）

```
    const set<string>
empty{"area","base","br","col","embed","hr","im
g","input",

"link","meta","source","track","wbr"};
```

改後（對照修改）

```
const set<string> empty{};
```

已試編並執行的變體測例：

輸入

```
<br></br>
```

輸出

```
VALID
```

04 求最大巢狀深度：改前 / 改後與測例

在 p 宣告後加 size_t deepest = 0; push 後加 if (stack.size() > deepest) deepest = stack.size(); 輸出行改為 if (valid) cout << deepest << "\n"; else cout << "INVALID\n"; 若同時改一行一案，每案開頭還需 deepest = 0;。

第 11 行：在此行後插入

改前（原碼）

```
size_t p = 0;
```

改後（對照修改）

```
size_t deepest = 0;
```

第 101 行：在此行後插入

改前（原碼）

```
        stack.push_back(tag);
```

改後（對照修改）

```
if (stack.size() > deepest) deepest =
stack.size();
```

第 127 行：用改後取代此行 / 區塊

改前（原碼）

```
    cout << (valid ? "VALID\n" : "INVALID\n");
```

改後（對照修改）

```
if (valid) cout << deepest << "\n"; else cout
<< "INVALID\n";
```

已試編並執行的變體測例：

輸入

```
<a><b></b></a>
```

輸出

```
2
```

05 首行 T，其後一行一案：改前 / 改後與測例

第 121–127 行：用改後取代此行 / 區塊

改前（原碼）

```
    s.assign(istreambuf_iterator<char>(cin),
{});
    // 讀取裝置失敗與文件語法非法不同：前者才用 exit
1。
    if (cin.bad()) { cerr << "input error\n";
return 1; }
    bool valid = false;
    // 語法錯誤也是分類結果 INVALID，所以這裡捕捉例
外，不把它當程式崩潰。
    try { valid = check(); } catch (const
runtime_error&) {}
    cout << (valid ? "VALID\n" : "INVALID\n");
```

改後（對照修改）

```
int T; if (!(cin >> T) || T < 0) return 1;
getline(cin, s);
if (s.find_first_not_of(" \t\r") !=
string::npos) return 1;
for (int tc = 0; tc < T; ++tc) {
    if (!getline(cin, s)) return 1;
    p = 0; bool valid = false;
    try { valid = check(); } catch (const
runtime_error&) {}
    cout << (valid ? "VALID\n" : "INVALID\n");
}
if (cin.bad()) return 1;
```

已試編並執行的變體測例：

輸入

```
2
<p/>
<p>
```

輸出

```
VALID
INVALID
```

02 遞迴下降運算式

題目內容

> Ch2 recursive descent / AST 的延伸模擬題；乘除、括號與一元正負不在原 ac 文法。

請讀入每行運算式，依通常優先序計算結果，或依題目指定改為 postfix / AST。一元正負優先於乘除，乘除優先於加減；同級二元左結合。括號可覆寫優先序。number 允許 .5、10.、十進位小數及 e/E 科學記號；ID 符合 [A–Za–z_][A–Za–z_0–9]*。求值模式不允許變數。

輸入規格

每行一個完整運算式到 EOF，沒有 T；完全空輸入是零案例，但空白行本身是非法運算式。

輸出規格

預設 eval：每行印 15 位有效數字的 long double 結果。postfix：以一個空白分隔 token。一元正負用 pos/neg。ast：印前序括號表示。

限制與適用範圍

嵌套括號 / 一元鏈最多 256；不支援 %、^、函式或隱含乘法。非法字元、缺 operand、零除、非有限值：stdout 空、stderr 訊息、exit 1。

Code 檔案名稱：`expression.cpp` （exam/short/expression.cpp）

```
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror expression.cpp -o main
./main < input.txt
```

題目範例與邊界測例

輸入

```
(2+3)*4
100-30-1
8/2/2
-2*-3
.5 + 1e-2
```

輸出

```
20
69
2
6
0.51
```

邊界測例 01

輸入

```
1+2*3
```

輸出

```
7
```

邊界測例 02

輸入

```
(2+3)*4
```

輸出

```
20
```

邊界測例 03

輸入

```
100-30-1
```

輸出

```
69
```

邊界測例 04

輸入

```
8/2/2
```

輸出

```
2
```

邊界測例 05

輸入

```
-2*-3
```

輸出

```
6
```

邊界測例 06

輸入

```
.5+1e-2
```

輸出

```
0.51
```

邊界測例 07

輸入

```
1/0
```

輸出

```
stdout：空
stderr：invalid expression
exit 1
```

邊界測例 08

輸入

```
2(3)
```

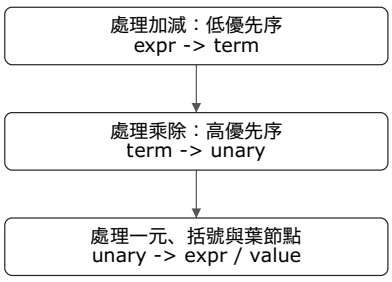
輸出

```
stdout：空
stderr：invalid expression
exit 1
```

邏輯與手動追蹤

expr 迴圈處理 + −，呼叫 term；term 迴圈處理 * /，呼叫 unary。unary 讀數字、變數或括號；同級二元由左往右合併。eval 不接受變數；括號 / 一元鏈嵌套上限 256。

- 100−30−1：先 100−30，再 −1，結果 69。
- a+b*c：AST 是 (+ a (* b c))；postfix 是 a b c * +。
- −2*−3：讀 2 → neg → 3 → neg → *，結果 6。
- 2(3) 並非隱含乘法，會因多餘內容拒絕。



方框：處理步驟　箭頭：執行順序或呼叫關係

題目變動時：先查修改位置

行號以含註解的 .cpp 原檔為準。每個變體獨立套用；多項一起改時，重新定位，不要沿用位移後的行號。表格列修改索引，詳細改前 / 改後與測例在後文列出。

題目要求	原檔行號	完整改法
只輸出 postfix	12	修改範例 01（後文）
只輸出 AST	12	修改範例 02（後文）

題目要求	原檔行號	完整改法
固定小數點後 2 位	106	修改範例 03（後文）
所有模式均禁止變數	51	修改範例 04（後文）
每次除法結果向零截斷	85	修改範例 05（後文）

---

01 只輸出 postfix：改前 / 改後與測例

const string MODE = "postfix";

第 12 行：用改後取代此行 / 區塊

改前（原碼）

```
const string MODE = "eval";
```

改後（對照修改）

```
const string MODE = "postfix";
```

已試編並執行的變體測例：

輸入

```
a+b*c
```

輸出

```
a b c * +
```

---

02 只輸出 AST：改前 / 改後與測例

const string MODE = "ast";

第 12 行：用改後取代此行 / 區塊

改前（原碼）

```
const string MODE = "eval";
```

改後（對照修改）

```
const string MODE = "ast";
```

已試編並執行的變體測例：

輸入

```
a+b*c
```

輸出

```
(+ a (* b c))
```

03 固定小數點後 2 位：改前 / 改後與測例

ostringstream out; out << fixed << setprecision(2);

第 106 行：用改後取代此行 / 區塊

改前（原碼）

```
    ostringstream out; out << setprecision(15);
```

改後（對照修改）

```
ostringstream out; out << fixed <<
setprecision(2);
```

已試編並執行的變體測例：

輸入

```
1/4
```

輸出

```
0.25
```

---

04 所有模式均禁止變數：改前 / 改後與測例

在這行後插入 if (id) fail(); ; eval 原本已禁止。

第 51 行：在此行後插入

改前（原碼）

```
    bool id = letter(c);
```

改後（對照修改）

```
if (id) fail();
```

已試編並執行的變體測例：

輸入

```
x+1
```

輸出

```
stdout：空
stderr：invalid expression
exit 1
```

---

05 每次除法結果向零截斷：改前 / 改後與測例

第 85 行：用改後取代此行 / 區塊

改前（原碼）

```
        if (op == '/') { if (b.number == 0)
fail(); a.number /= b.number; }
```

改後（對照修改）

```
        if (op == '/') { if (b.number == 0)
fail(); a.number = trunc(a.number /
b.number); }
```

已試編並執行的變體測例：

輸入

```
-7/2
```

輸出

```
-3
```

03 Scanner：沿用現有短程式

題目內容

> Ch3 scanner / longest match 的延伸模擬題。教材 HW3 要擴充 ac 的 f/float、i/int、scientific number；本份是一般 C-like token 題，規格不同。

請對一份類 C 程式做詞彙分析，依序列出 keyword、identifier、number、string、char、operator 與 punctuation。忽略空白、單行 // 註解及非巢狀 /* */ 註解。先讀完整 identifier 再查保留字，operator 採最長匹配。quoted literal 中的註解符號不可當作註解，跳脫引號不得提早終止 literal。

輸入規格

整份 stdin 到 EOF；ID 可含底線；number 允許 .5、10.、e/E。keyword 名單及 operator 名單以檔案常數為準。

輸出規格

每個 token 一行：類型、空白、std::quoted 包裝的原始 lexeme；最後一行 EOF。

限制與適用範圍

不是完整 C/C++ lexer：沒有 preprocessing、raw string、數字 suffix 或完整字元 escape 語意；CHAR 只允許單 byte / 單簡單 escape。遇未閉註解 / 引號或非法 byte，exit 1 且 stdout 空。

Code 檔案名稱：`lexer.cpp` （exam/lexer.cpp）

```
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror lexer.cpp -o main
./main < input.txt
```

題目範例與邊界測例

輸入

```
int sum=12; // skip
float x=1.2e-3; /*skip*/ sum+=2; print("a>b");
```

輸出

```
KW "int"
ID "sum"
OP "="
NUM "12"
PUNCT ";"
KW "float"
ID "x"
OP "="
NUM "1.2e-3"
PUNCT ";"
ID "sum"
OP "+="
NUM "2"
PUNCT ";"
KW "print"
PUNCT "("
STRING "\"a>b\""
PUNCT ")"
PUNCT ";"
EOF
```

邊界測例 01

輸入

```
intx >>= 2
```

輸出

```
ID "intx"
OP ">>="
NUM "2"
EOF
```

邊界測例 02

輸入

```
int x; // hi
```

輸出

```
KW "int"
ID "x"
PUNCT ";"
EOF
```

邊界測例 03

輸入

```
"//x"
```

輸出

```
STRING "\"//x\""
EOF
```

邊界測例 04

輸入

```
1e+
```

輸出

```
stdout：空
stderr：error: missing exponent digits
exit 1
```

邊界測例 05

輸入

```
/* unfinished
```

輸出

```
stdout：空
stderr：error: unclosed block comment
exit 1
```

邏輯與手動追蹤

依序判定空白、註解、ID、number、literal、operator。讀完整 ID 才查 keyword；多字元 operator 由長到短比對；字串內的註解符號不當成註解。

- intx：先讀四個字元，查 keyword 失敗，因此為 ID，不拆 int + x。
- a>>=2：>>= 先於 >> / >，只有一個 operator token。
- "//x"：整段是 STRING，// 不觸發註解。
- –1e–2：前面的 – 是 OP，指數內的 – 是 NUM 的一部分。

題目變動時：先查修改位置

行號以含註解的 .cpp 原檔為準。每個變體獨立套用；多項一起改時，重新定位，不要沿用位移後的行號。表格列修改索引，詳細改前 / 改後與測例在後文列出。

題目要求	原檔行號	完整改法
增加保留字	23	修改範例 01（後文）
增加多字元運算子	25	修改範例 02（後文）
lexeme 不加引號	87	修改範例 03（後文）
不要印 EOF	90	修改範例 04（後文）
不區分 keyword，全部列為 ID	44	修改範例 05（後文）

01 增加保留字：改前 / 改後與測例

在 keywords 名單加入 "switch", "case" 等題目指定字。

第 23 行：用改後取代此行 / 區塊

改前（原碼）

```
        const std::set<std::string>
keywords{"int","float","double","char","bool","
void","if","else","while","for","return","print
"};
```

改後（對照修改）

```
        const std::set<std::string>
keywords{"switch","case","int","float","double"
,"char","bool","void","if","else","while","for"
,"return","print"};
```

已試編並執行的變體測例：

輸入

```
switch case
```

輸出

```
KW "switch"
KW "case"
EOF
```

02 增加多字元運算子：改前 / 改後與測例

在 ops 名單加入所需運算子；維持較長字串先列。這裡只分詞，不增加 parser 的運算規則。

第 25 行：用改後取代此行 / 區塊

改前（原碼）

```
        const std::vector<std::string>
ops{"<<=",">>=","++","--","==","!
=","<=",">=","&&","||","+=","-=","*=","/
=","%=","<<",">>","->","&=","|=","^=","::"};
```

改後（對照修改）

```
        const std::vector<std::string>
ops{"<<=",">>=","**","++","--","==","!
=","<=",">=","&&","||","+=","-=","*=","/
=","%=","<<",">>","->","&=","|=","^=","::"};
```

已試編並執行的變體測例：

輸入

```
a**b
```

輸出

```
ID "a"
OP "**"
ID "b"
EOF
```

03 lexeme 不加引號：改前 / 改後與測例

output << kind << ' ' << s.substr(start, p – start) << '\n';

第 87 行：用改後取代此行 / 區塊

改前（原碼）

```
            output << kind << ' ' <<
std::quoted(s.substr(start, p - start)) <<
'\n';
```

改後（對照修改）

```
output << kind << ' ' << s.substr(start, p -
start) << '\n';
```

已試編並執行的變體測例：

輸入

```
int x
```

輸出

```
KW int
ID x
EOF
```

04 不要印 EOF：改前 / 改後與測例

刪除這一行。

第 90 行：用改後取代此行 / 區塊

改前（原碼）

```
        output << "EOF\n";
```

改後（對照修改）

```
（刪除；不要貼這段提示文字）
```

已試編並執行的變體測例：

輸入

```
int
```

輸出

```
KW "int"
```

05 不區分 keyword，全部列為 ID：改前 / 改後與測例

第 44 行：用改後取代此行 / 區塊

改前（原碼）

```
                kind =
keywords.count(s.substr(start, p - start)) ?
"KW" : "ID";
```

改後（對照修改）

```
kind = "ID";
```

已試編並執行的變體測例：

輸入

```
int x
```

輸出

```
ID "int"
ID "x"
EOF
```

04 NFA 直接判定字串

題目內容

> Ch3 epsilon-closure 與 NFA 的字串判定模擬題；input 格式由這份手冊定義。

給定帶 epsilon edge 的 NFA，請對每個完整輸入字串判定是否接受。NFA 在同一時刻可同時位於多個 state，且不用消耗字元也可沿 epsilon edge 移動。每讀一個字元，對全部當前 state 做 move，再做 epsilon-closure。所有字元用完後，只要集合含至少一個接受 state 就接受。

輸入規格

先讀 n m start k；再讀 k 個 final state；再讀 alphabet；接著 m 條 from symbol to。state 是 0..n-1；# 是 epsilon。其後每行一個字串，空行是 epsilon 案例。

輸出規格

每案一行 ACCEPT / REJECT。非法 alphabet 字元直接拒絕該字串；非法 NFA 描述則 stderr、exit 1。

限制與適用範圍

1<=n<=256、0<=m<=65536；alphabet 非空、無重複、不能含 #；只接受單 byte 符號。本份不輸出 subset DFA 表。

Code 檔案名稱：`nfa_check.cpp` （exam/short/nfa_check.cpp）

```
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror nfa_check.cpp -o main
./main < input.txt
```

題目範例與邊界測例

輸入

```
6 7 0 1
5
bcd
0 # 1
0 # 4
1 b 2
2 c 3
3 # 1
3 # 4
4 d 5
d
bcd
bcbcd

bc
bd
bcdd
```

輸出

```
ACCEPT
ACCEPT
ACCEPT
REJECT
REJECT
REJECT
REJECT
```

邊界測例 01

輸入

```
3 2 0 1
2
a
0 # 1
1 a 2
a

aa
z
```

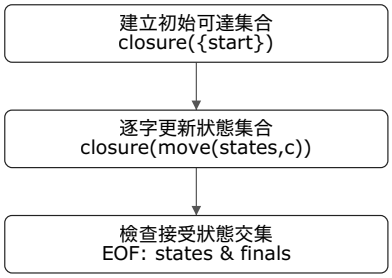
輸出

```
ACCEPT
REJECT
REJECT
REJECT
```

邏輯與手動追蹤

初始集合是 start 的 ε–closure。每讀一字元，先 move 再 ε–closure；讀完才檢查是否含接受狀態。空行代表空字串。這份只判定字串；要 DFA 表須用 nfa_to_dfa.cpp。

- 0 –#→ 1 –a→ 2(final)：初始 {0,1}；讀 a 後 {2}，接受。
- epsilon cycle：set 去重，只有新 state 進 todo，所以 closure 不會卡住。
- 先碰 final 再走死：必須等字串讀完，不能中途接受。



方框：處理步驟　箭頭：執行順序或呼叫關係

題目變動時：先查修改位置

行號以含註解的 .cpp 原檔為準。每個變體獨立套用；多項一起改時，重新定位，不要沿用位移後的行號。表格列修改索引，詳細改前 / 改後與測例在後文列出。

題目要求	原檔行號	完整改法
state 編號從 1 開始	31；40	修改範例 01（後文）
epsilon 用 @	20；37；44	修改範例 02（後文）
輸出 YES / NO	68	修改範例 03（後文）
印初始及每讀一字元後的 state 集合	55；63	修改範例 04（後文）
忽略字串前後空白	53	修改範例 05（後文）

01 state 編號從 1 開始：改前 / 改後與測例

valid 行改成 auto valid = [n](int state) { return state >= 1 && state <= n; }; resize 行改成 edges.resize(n + 1); ；輸入 start、finals、edges 皆維持 1–based。

第 31 行：用改後取代此行 / 區塊

改前（原碼）

```
        auto valid = [n](int state) { return
state >= 0 && state < n; };
```

改後（對照修改）

```
auto valid = [n](int state) { return state >= 1
&& state <= n; };
```

第 40 行：用改後取代此行 / 區塊

改前（原碼）

```
        edges.resize(n);
```

改後（對照修改）

```
edges.resize(n + 1);
```

已試編並執行的變體測例：

輸入

```
2 1 1 1
2
a
1 a 2
a
b
```

輸出

```
ACCEPT
REJECT
```

02 epsilon 用 @：改前 / 改後與測例

這三行的 '#' 與 "#" 改成 '@' 與 "@"。

第 20 行：用改後取代此行 / 區塊

改前（原碼）

```
        for (int to : edges[from]['#']) if
(states.insert(to).second) todo.push_back(to);
```

改後（對照修改）

```
        for (int to : edges[from]['@']) if
(states.insert(to).second) todo.push_back(to);
```

第 37 行：用改後取代此行 / 區塊

改前（原碼）

```
        if (!(cin >> alphabet) ||
alphabet.find('#') != string::npos ||
```

改後（對照修改）

```
        if (!(cin >> alphabet) ||
alphabet.find('@') != string::npos ||
```

第 44 行：用改後取代此行 / 區塊

改前（原碼）

```
                (symbol != "#" &&
alphabet.find(symbol[0]) == string::npos))
fail();
```

改後（對照修改）

```
                (symbol != "@" &&
alphabet.find(symbol[0]) == string::npos))
fail();
```

已試編並執行的變體測例：

輸入

```
3 2 0 1
2
a
0 @ 1
1 a 2
a
```

輸出

```
ACCEPT
```

03 輸出 YES / NO：改前 / 改後與測例

將 "ACCEPT\n" / "REJECT\n" 改成 "YES\n" / "NO\n"。

第 68 行：用改後取代此行 / 區塊

改前（原碼）

```
            cout << (accepted ? "ACCEPT\n" :
"REJECT\n");
```

改後（對照修改）

```
            cout << (accepted ? "YES\n" :
"NO\n");
```

已試編並執行的變體測例：

輸入

```
2 1 0 1
1
a
0 a 1
a
```

輸出

```
YES
```

04 印初始及每讀一字元後的 state 集合：改前 / 改後與測例

第 55 行：在此行後插入

改前（原碼）

```
            States states = closure({start});
```

改後（對照修改）

```
for (int state : states) cout << state << ' ';
cout << '\n';
```

第 63 行：在此行後插入

改前（原碼）

```
                states = closure(next);
```

改後（對照修改）

```
for (int state : states) cout << state << ' ';
cout << '\n';
```

已試編並執行的變體測例：

輸入

```
3 2 0 1
2
a
0 # 1
1 a 2
a
```

輸出

```
0 1
2
ACCEPT
```

05 忽略字串前後空白：改前 / 改後與測例

第 53 行：用改後取代此行 / 區塊

改前（原碼）

```
            if (!word.empty() && word.back() ==
'\r') word.pop_back();
```

改後（對照修改）

```
size_t first = word.find_first_not_of(" \t\r");
word = first == string::npos ? "" :
word.substr(first, word.find_last_not_of("
\t\r") - first + 1);
```

已試編並執行的變體測例：

輸入

```
2 1 0 1
1
a
0 a 1
  a
```

輸出

```
ACCEPT
```

05 HW5 固定四狀態 DFA

題目內容

> 教材原題（Ch3 PPT p.55、p.58）：Write NFA, and then DFA that recognizes the tokens defined by the following regular expression；5(d)=(bc)*d，5(a)=(a|(bc)*d)+。以下字串 I/O 是另外定義的程式練習。

請以 DFA 判定字串是否屬於 5(a) 或 5(d) 的語言。5(a) 是一段或多段 a 或 (bc)*d 的串接；5(d) 是零次或多次 bc 後接單一 d。所有字元必須用完才可接受。原作業若要求畫 NFA / subset DFA，另外用 automata_homework.cpp 的 --table；本短版只給辨識結果。

輸入規格

HW5A=true 選 5(a)，改 false 選 5(d)。其後每行一個字串到 EOF，沒有 T；空行代表空字串。

輸出規格

每案印 ACCEPT / REJECT。大小寫敏感，僅題目語言內的字母可接受。

限制與適用範圍

不支援任意正則表示式輸入；這是固定兩個語言的四個可達 state 加 dead state。

Code 檔案名稱：`hw5_check.cpp`（exam/short/hw5_check.cpp）

```
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror hw5_check.cpp -o main
./main < input.txt
```

題目範例與邊界測例

輸入

```
a
d
ad
bcd
bcbcd
abcbcdad
b
bc
abca
bcdd
```

輸出

```
REJECT
ACCEPT
ACCEPT
ACCEPT
ACCEPT
ACCEPT
ACCEPT
REJECT
REJECT
REJECT
ACCEPT
```

邊界測例 01

輸入

```
a
d
ad
bcd
bc
dd
z
```

輸出

```
REJECT
ACCEPT
ACCEPT
ACCEPT
ACCEPT
REJECT
ACCEPT
REJECT
```

邏輯與手動追蹤

state 0 起始、1 剛讀 b、2 剛讀 bc、3 接受、–1 死狀態。1 只能接 c；2 只能接 b 或 d；HW5(a) 的 3 可接下一組，HW5(d) 的 3 再讀字元就拒絕。

- 5(a)：a、d、ad、bcd、dd 接受；bc、空字串拒絕。
- 5(d)：d、bcd、bcbcd 接受；a、dd、bc 拒絕。
- bcd：0 –b→ 1 –c→ 2 –d→ 3；最後在 3，接受。

題目變動時：先查修改位置

行號以含註解的 .cpp 原檔為準。每個變體獨立套用；多項一起改時，重新定位，不要沿用位移後的行號。表格列修改索引，詳細改前 / 改後與測例在後文列出。

題目要求	原檔行號	完整改法
改解 HW5(d)	6	修改範例 01（後文）
輸出 YES / NO	30	修改範例 02（後文）
HW5(a) 外層 + 改成 *	30	修改範例 03（後文）
ASCII 大小寫不敏感	13	修改範例 04（後文）
輸出 Case 1: 結果	9；30	修改範例 05（後文）

---

01 改解 HW5(d)：改前 / 改後與測例

const bool HW5A = false;

第 6 行：用改後取代此行 / 區塊

改前（原碼）

```
const bool HW5A = true;
```

改後（對照修改）

```
const bool HW5A = false;
```

已試編並執行的變體測例：

輸入

```
d
dd
bcd
```

輸出

```
ACCEPT
REJECT
ACCEPT
```

02 輸出 YES / NO：改前 / 改後與測例

將 "ACCEPT\n" / "REJECT\n" 改成 "YES\n" / "NO\n"。

第 30 行：用改後取代此行 / 區塊

改前（原碼）

```
        cout << (state == 3 ? "ACCEPT\n" :
"REJECT\n");
```

改後（對照修改）

```
        cout << (state == 3 ? "YES\n" :
"NO\n");
```

已試編並執行的變體測例：

輸入

```
a
b
```

輸出

```
YES
NO
```

03 HW5(a) 外層 + 改成 *：改前 / 改後與測例

HW5A 保持 true；輸出條件 state == 3 改成 (state == 3 ‖ word.empty())，使空字串也接受。

第 30 行：用改後取代此行 / 區塊

改前（原碼）

```
        cout << (state == 3 ? "ACCEPT\n" :
"REJECT\n");
```

改後（對照修改）

```
        cout << ((state == 3 || word.empty()) ?
"ACCEPT\n" : "REJECT\n");
```

已試編並執行的變體測例：

輸入

```
a
bc
```

輸出

```
ACCEPT
ACCEPT
REJECT
```

04 ASCII 大小寫不敏感：改前 / 改後與測例

第 13 行：在此行後插入

改前（原碼）

```
        if (!word.empty() && word.back() ==
'\r') word.pop_back();
```

改後（對照修改）

```
for (char& c : word) if (c >= 'A' && c <= 'Z')
c += 'a' - 'A';
```

已試編並執行的變體測例：

輸入

```
BCD
A
```

輸出

```
ACCEPT
ACCEPT
```

05 輸出 Case 1: 結果：改前 / 改後與測例

第 9 行：在此行後插入

改前（原碼）

```
    string word;
```

改後（對照修改）

```
int case_no = 0;
```

第 30 行：用改後取代此行 / 區塊

改前（原碼）

```
        cout << (state == 3 ? "ACCEPT\n" :
"REJECT\n");
```

改後（對照修改）

```
        cout << "Case " << ++case_no << ": " <<
(state == 3 ? "ACCEPT\n" : "REJECT\n");
```

已試編並執行的變體測例：

輸入

```
a
b
```

輸出

```
Case 1: ACCEPT
Case 2: REJECT
```

06 ac 邊解析邊產 dc

題目內容

> 教材原題（Ch2 PPT p.39–46）：HW1 對 i x x=100 x=x+30+1 p x 建 parse tree；HW2(a) 建 AST、(b) 判斷語意與所需語言變更、(c) 顯示產碼；HW3 讓 f/float、i/int 均可宣告，number 可帶 optionally signed exponent。本檔只解產碼與型別檢查部分。

請把 ac 程式編譯為 dc stack code。所有宣告在敘述前，型別為 int / float；identifier 為一個小寫字母，i/f/p 是保留字。敘述為 id=val{(+|−)val} 或 p id。val 為 variable 或非負 number。加減左結合。檢查重複宣告、未宣告及 float→int 指定；int→float 可提升。科學記號用精確字串移動小數點轉為 dc 可讀的小數。

輸入規格

整份 stdin 到 EOF；沒有分號或 T。f 或 float、i 或 int 均可。小數的點後必須有數字，指數字母限 lowercase e，可有正負。

輸出規格

只印 dc target code，不直接印數值答案。非法 ac 時 stdout 空、stderr 訊息、exit 1。

限制與適用範圍

帶 . 或 e 的 number 在本實作一律 float；exponent 絕對值與 mantissa 長度 <=10000。無乘除、括號、unary、註解。短版無 parse tree / AST / folding，這些要求使用 ac_homework.cpp。教材對 HW2(b) 的 No 與型別規則有疑義，詳見既有作業校讀。

Code 檔案名稱：`ac_dc.cpp`　(exam/short/ac_dc.cpp)

```
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror ac_dc.cpp -o main
./main < input.txt
```

題目範例與邊界測例

輸入

```
i x
x=100
x=x+30+1
p x
```

輸出

```
100
sx
0 k
lx
30
+
1
+
sx
0 k
lx
p
si
```

邊界測例 01

輸入

```
i x x=2 px
```

輸出

```
2
sx
0 k
lx
p
si
```

邊界測例 02

輸入

```
f x x=1e-2 px
```

輸出

```
0.01
sx
0 k
lx
p
si
```

邊界測例 03

輸入

```
i x x=1.2
```

輸出

```
stdout：空
stderr：invalid ac program
exit 1
```

邊界測例 04

輸入

```
p x
```

輸出

```
stdout：空
stderr：invalid ac program
exit 1
```

邊界測例 05

輸入

```
i x i x
```

輸出

```
stdout：空
stderr：invalid ac program
exit 1
```

邏輯與手動追蹤

scan 成 token；宣告填 types；value 回傳 leaf code 與是否 float。解析加減時直接輸出 postfix；int→float 印 5 k，assign 印 s<變數>，print 印 l<變數>/p/si。要樹或常數摺疊，用 ac_homework.cpp。

- x=100：100 → sx → 0 k；p x：lx → p → si。
- i a f b b=a+3.2：la → 5 k → 3.2 → +；提升必須在對應 operand 後。
- 123e-22 → 0.00000000000000000000123，不能先轉 double 再印。
- i x x=1e0：數值雖為 1，來源型別是 float，所以拒絕指定給 int。

題目變動時：先查修改位置

行號以含註解的 .cpp 原檔為準。每個變體獨立套用；多項一起改時，重新定位，不要沿用位移後的行號。表格列修改索引，詳細改前 / 改後與測例在後文列出。

題目要求	原檔行號	完整改法
只允許 i / f 短宣告	43；44；45	修改範例 01（後文）
科學記號也接受 E	35；57；98	修改範例 02（後文）
浮點運算 scale 改成 10	128；131；136	修改範例 03（後文）
合法程式只輸出 OK	140	修改範例 04（後文）
assignment 後不要產生 scale reset	138	修改範例 05（後文）

---

01 只允許 i / f 短宣告：改前 / 改後與測例

刪除 float / int 完整字的兩行判定；第三行開頭 else if 改成 if。

第 43 行：用改後取代此行 / 區塊

改前（原碼）

```
            if (kind == 'f' && s.compare(start,
5, "float") == 0) j = start + 5;
```

改後（對照修改）

```
（刪除；不要貼這段提示文字）
```

第 44 行：用改後取代此行 / 區塊

改前（原碼）

```
            else if (kind == 'i' &&
s.compare(start, 3, "int") == 0) j = start + 3;
```

改後（對照修改）

```
（刪除；不要貼這段提示文字）
```

第 45 行：用改後取代此行 / 區塊

改前（原碼）

```
            else if ((kind >= 'a' && kind <=
'z') || kind == '=' || kind == '+' || kind ==
'-') {}
```

改後（對照修改）

```
            if ((kind >= 'a' && kind <= 'z') ||
kind == '=' || kind == '+' || kind == '-') {}
```

已試編並執行的變體測例：

輸入

```
i x x=1 px
```

輸出

```
1
sx
0 k
lx
p
si
```

02 科學記號也接受 E：改前 / 改後與測例

條件改成 if (j < s.size() && (s[j] == 'e' ‖ s[j] == 'E')) { find 改成 text.find_first_of("eE"); find_first_of(".e") 改成 find_first_of(".eE")。

第 35 行：用改後取代此行 / 區塊

改前（原碼）

```
            if (j < s.size() && s[j] == 'e') {
```

改後（對照修改）

```
if (j < s.size() && (s[j] == 'e' || s[j] ==
'E')) {
```

第 57 行：用改後取代此行 / 區塊

改前（原碼）

```
    size_t e = text.find('e');
```

改後（對照修改）

```
size_t e = text.find_first_of("eE");
```

第 98 行：用改後取代此行 / 區塊

改前（原碼）

```
    return {decimal(text) + "\n",
text.find_first_of(".e") != string::npos};
```

改後（對照修改）

```
    return {decimal(text) + "\n",
text.find_first_of(".eE") != string::npos};
```

已試編並執行的變體測例：

輸入

```
f x x=1E-2 px
```

輸出

```
0.01
sx
0 k
lx
p
si
```

03 浮點運算 scale 改成 10：改前 / 改後與測例

三行的 "5 k\n" 都改成 "10 k\n"；assignment 的 "0 k\n" 保留。

第 128 行：用改後取代此行 / 區塊

改前（原碼）

```
                if (!left.second &&
right.second) out << "5 k\n";
```

改後（對照修改）

```
                if (!left.second &&
right.second) out << "10 k\n";
```

第 131 行：用改後取代此行 / 區塊

改前（原碼）

```
                if (left.second && !
right.second) out << "5 k\n";
```

改後（對照修改）

```
                if (left.second && !
right.second) out << "10 k\n";
```

第 136 行：用改後取代此行 / 區塊

改前（原碼）

```
            if (target_float && !left.second)
out << "5 k\n";
```

改後（對照修改）

```
            if (target_float && !left.second)
out << "10 k\n";
```

已試編並執行的變體測例：

輸入

```
f x x=1 px
```

輸出

```
1
10 k
sx
0 k
lx
p
si
```

04 合法程式只輸出 OK：改前 / 改後與測例

第 140 行：用改後取代此行 / 區塊

改前（原碼）

```
        cout << out.str(); return cout ? 0 : 1;
```

改後（對照修改）

```
cout << "OK\n"; return cout ? 0 : 1;
```

已試編並執行的變體測例：

輸入

```
i x x=2 px
```

輸出

```
OK
```

05 assignment 後不要產生 scale reset：改前 / 改後與測例

第 138 行：用改後取代此行 / 區塊

改前（原碼）

```
            out << 's' << target << "\n0 k\n";
```

改後（對照修改）

```
            out << 's' << target << "\n";
```

已試編並執行的變體測例：

輸入

```
i x x=2 px
```

輸出

```
2
sx
lx
p
si
```

07 Stack VM：沿用現有短程式

題目內容

> Ch1 interpreter / Ch2 dc 產碼的延伸模擬題；指令名稱由本題定義，不是 GNU dc。

請執行 stack-based 虛擬機指令。PUSH number 把常數入 stack；STORE name pop 並保存；LOAD name 複製變數值。ADD/SUB/MUL/DIV 先取右 operand b，再取左 a，計算 a op b。NEG 取反，DUP 複製頂端，PRINT pop 並輸出。未知指令、未定義變數、underflow、零除或非有限值均視為錯誤。

輸入規格

每行一條指令到 EOF；空行忽略。PUSH/LOAD/STORE 恰有一個參數，其餘沒有參數；變數名採一般 ID 規則。

輸出規格

每次 PRINT 一行，15 位有效數字。整份成功後才發出累積輸出；任何錯誤 stdout 空、stderr 訊息、exit 1。

限制與適用範圍

long double，非任意精度；沒有跳躍、條件、函式或字串。EOF 可留未使用 stack；PRINT 與 dc p 的 pop 規則不同。

Code 檔案名稱：`stack_vm.cpp` (exam/stack_vm.cpp)

```
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror stack_vm.cpp -o main
./main < input.txt
```

題目範例與邊界測例

輸入

```
PUSH 100
STORE x
LOAD x
PUSH 30
PUSH 1
ADD
ADD
STORE x
LOAD x
PRINT
```

輸出

```
131
```

邊界測例 01

輸入

```
PUSH 8
PUSH 2
SUB
PRINT
```

輸出

```
6
```

邊界測例 02

輸入

```
PUSH 2
DUP
MUL
NEG
PRINT
```

輸出

```
-4
```

邊界測例 03

輸入

```
PUSH 1
PUSH 0
DIV
PRINT
```

輸出

```
stdout：空
stderr：error: division by zero
exit 1
```

邊界測例 04

輸入

```
LOAD x
```

輸出

```
stdout：空
stderr：error: undefined variable x
exit 1
```

邊界測例 05

輸入

```
PRINT
```

輸出

```
stdout：空
stderr：error: stack underflow
exit 1
```

邏輯與手動追蹤

vector 當 stack，map 存變數。二元先 pop 右 b，再 pop 左 a，算 a op b；STORE 與 PRINT 都 pop。空 stack、未定義 LOAD、零除均報錯。

- PUSH 8 / PUSH 2 / SUB / PRINT：8−2=6，不能算成 2−8。
- PUSH 2 / DUP / MUL / NEG / PRINT：2*2 再取負，得 −4。
- PRINT 後 stack 少一項；若題目要求保留，要改成 peek。

題目變動時：先查修改位置

行號以含註解的 .cpp 原檔為準。每個變體獨立套用；多項一起改時，重新定位，不要沿用位移後的行號。表格列修改索引，詳細改前 / 改後與測例在後文列出。

題目要求	原檔行號	完整改法
PRINT 保留 stack 頂端	54	修改範例 01（後文）
固定小數點後 2 位	29	修改範例 02（後文）
PRINT 指令改名為 OUT	54	修改範例 03（後文）
指令名稱忽略 ASCII 大小寫	38	修改範例 04（後文）

題目要求	原檔行號	完整改法
PRINT 不移除值，DUP 後連續列印	54	修改範例 05（後文）

01 PRINT 保留 stack 頂端：改前 / 改後與測例

} else if (op == "PRINT") { if (stack.empty()) throw std::runtime_error("stack underflow"); const auto value = stack.back(); output << (value == 0 ? 0 : value) << '\n'; }

第 54 行：用改後取代此行 / 區塊

改前（原碼）

```
            } else if (op == "PRINT") { const
auto value = pop(); output << (value == 0 ? 0 :
value) << '\n'; }
```

改後（對照修改）

```
} else if (op == "PRINT") {
    if (stack.empty()) throw
std::runtime_error("stack underflow");
    const auto value = stack.back();
    output << (value == 0 ? 0 : value) << '\n';
}
```

已試編並執行的變體測例：

輸入

```
PUSH 2
PRINT
PRINT
```

輸出

```
2
2
```

02 固定小數點後 2 位：改前 / 改後與測例

把 output << std::setprecision(15); 改成 output << std::fixed << std::setprecision(2);。

第 29 行：用改後取代此行 / 區塊

改前（原碼）

```
        std::ostringstream output;
output.imbue(std::locale::classic()); output <<
std::setprecision(15);
```

改後（對照修改）

```
        std::ostringstream output;
output.imbue(std::locale::classic()); output <<
std::fixed << std::setprecision(2);
```

已試編並執行的變體測例：

輸入

```
PUSH .25
PRINT
```

輸出

```
0.25
```

03 PRINT 指令改名為 OUT：改前 / 改後與測例

把該行 op == "PRINT" 改成 op == "OUT"。

第 54 行：用改後取代此行 / 區塊

改前（原碼）

```
            } else if (op == "PRINT") { const
auto value = pop(); output << (value == 0 ? 0 :
value) << '\n'; }
```

改後（對照修改）

```
            } else if (op == "OUT") { const
auto value = pop(); output << (value == 0 ? 0 :
value) << '\n'; }
```

已試編並執行的變體測例：

輸入

```
PUSH 8
OUT
```

輸出

```
8
```

04 指令名稱忽略 ASCII 大小寫：改前 / 改後與測例

第 38 行：在此行後插入

改前（原碼）

```
            if (!(input >> op)) continue;
```

改後（對照修改）

```
for (char& c : op) if (c >= 'a' && c <= 'z') c
-= 'a' - 'A';
```

已試編並執行的變體測例：

輸入

```
push 2
print
```

輸出

```
2
```

05 PRINT 不移除值，DUP 後連續列印：改前 / 改後與測例

第 54 行：用改後取代此行 / 區塊

改前（原碼）

```
            } else if (op == "PRINT") { const
auto value = pop(); output << (value == 0 ? 0 :
value) << '\n'; }
```

改後（對照修改）

```
} else if (op == "PRINT") {
    if (stack.empty()) throw
std::runtime_error("stack underflow");
    const auto value = stack.back();
    output << (value == 0 ? 0 : value) << '\n';
}
```

已試編並執行的變體測例：

輸入

```
PUSH 2
DUP
PRINT
PRINT
```

輸出

```
2
2
```

08 LL(1)：較低優先補充

題目內容

> predict 的通用延伸模擬題；正式 FIRST/FOLLOW 術語預告於 Ch4，不能視為已確認 Ch1–3 考題。

給定 CFG，請計算各 nonterminal 的 FIRST / FOLLOW、各 production 的 SELECT，並建立 LL(1) 解析表。FIRST / FOLLOW 必須反覆傳播至固定點。RHS 可空時將 LHS 的 FOLLOW 加入 SELECT。同格若有多條 production，列出 conflict，判定非 LL(1)。無衝突時另可用 predictive stack 判定 token 序列。

輸入規格

先讀 P（1..1000），再讀 P 行 A -> X Y。每行只含一條 alternative；# 必須單獨表示 epsilon，$ 保留為 EOF。start 是第一個 LHS。--check 模式才讀其後每行空白分隔的 token sequence。

輸出規格

預設列 FIRST/FOLLOW/SELECT、非空表格 cell、CONFLICT 與 LL1 YES/NO。--check 每案印 ACCEPT/REJECT；不由字串原始字元自動 tokenize。

限制與適用範圍

不消除左遞迴、不做 left factoring，也不判定任意 CFG 是否無歧義；parse 步數 <=1000000。有 LL(1) conflict 時不任選規則繼續。

Code 檔案名稱：`ll1.cpp` (exam/ll1.cpp)

```
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Werror ll1.cpp -o main
./main < input.txt
```

題目範例與邊界測例

輸入

```
4
E -> T R
R -> + T R
R -> #
T -> num
```

輸出

```
FIRST(E)={num} FOLLOW(E)={$}
FIRST(R)={#,+} FOLLOW(R)={$}
FIRST(T)={num} FOLLOW(T)={$,+}
SELECT(1)={num}
SELECT(2)={+}
SELECT(3)={$}
SELECT(4)={num}
M[E,num]=1
M[R,$]=3
M[R,+]=2
M[T,num]=4
LL1 YES
```

邊界測例 01

輸入

```
2
S -> a S
S -> #
```

輸出

```
FIRST(S)={#,a} FOLLOW(S)={$}
SELECT(1)={a}
SELECT(2)={$}
M[S,$]=2
M[S,a]=1
LL1 YES
```

邏輯與手動追蹤

FIRST 與 FOLLOW 反覆合併至固定點；nullable RHS 將 FOLLOW 加進 SELECT。用 SELECT 填表，同格多條規則就是 conflict；解析用 stack 與 lookahead 查表。# 單獨代表 ε。

- S –> a S | #：FIRST(S)={#,a}、FOLLOW(S)={$}。
- SELECT(S–>#)={$}，所以 EOF 時才選 epsilon，而不是看到任何字元都選。
- A –> B C：若 C nullable，FOLLOW(A) 也要傳給 FOLLOW(B)。
- 展開 S –> a S 時先 push S 再 push a，讓 a 先被處理。

題目變動時：先查修改位置

行號以含註解的 .cpp 原檔為準。每個變體獨立套用；多項一起改時，重新定位，不要沿用位移後的行號。表格列修改索引，詳細改前 / 改後與測例在後文列出。

題目要求	原檔行號	完整改法
judge 不傳參數，只判定字串	35	修改範例 01（後文）
只印 FIRST 與 FOLLOW	96–103	修改範例 02（後文）
字串判定輸出 YES / NO	129	修改範例 03（後文）
指定 S 為 start，不使用第一個 LHS	68；112	修改範例 04（後文）
只輸出是否為 LL(1)	93–103	修改範例 05（後文）

01 judge 不傳參數，只判定字串：改前 / 改後與測例

const std::string mode = argc == 1 ? "--check" : argv[1];

第 35 行：用改後取代此行 / 區塊

改前（原碼）

```
        const std::string mode = argc == 1 ?
"--table" : argv[1];
```

改後（對照修改）

```
const std::string mode = argc == 1 ? "--
check" : argv[1];
```

已試編並執行的變體測例：

輸入

```
2
S -> a S
S -> #
a a

b
```

輸出

```
ACCEPT
ACCEPT
REJECT
```

02 只印 FIRST 與 FOLLOW：改前 / 改後與測例

刪除這段 SELECT / table / LL1 輸出；FIRST/FOLLOW 兩行不刪，前面的集合計算不刪。

第 96–103 行：用改後取代此行 / 區塊

改前（原碼）

```
            for (std::size_t i = 0; i <
rules.size(); ++i) { std::cout << "SELECT(" <<
i + 1 << ")="; print_set(select[i]); std::cout
<< '\n'; }
            for (const auto& cell : table) {
                std::cout << "M[" <<
cell.first.first << ',' << cell.first.second <<
"]=";
                for (std::size_t i = 0; i <
cell.second.size(); ++i) std::cout << (i ?
"," : "") << cell.second[i] + 1;
                if (cell.second.size() > 1)
std::cout << " CONFLICT";
                std::cout << '\n';
            }
            std::cout << (ll1 ? "LL1 YES\n" :
"LL1 NO\n");
```

改後（對照修改）

```
（刪除；不要貼這段提示文字）
```

已試編並執行的變體測例：

輸入

```
2
S -> a S
S -> #
```

輸出

```
FIRST(S)={#,a} FOLLOW(S)={$}
```

03 字串判定輸出 YES / NO：改前 / 改後與測例

將 "ACCEPT\n" / "REJECT\n" 改成 "YES\n" / "NO\n"。

第 129 行：用改後取代此行 / 區塊

改前（原碼）

```
                std::cout << (accepted && pos
== words.size() ? "ACCEPT\n" : "REJECT\n");
```

改後（對照修改）

```
                std::cout << (accepted && pos
== words.size() ? "YES\n" : "NO\n");
```

此變體執行參數：`./main --check`

已試編並執行的變體測例：

輸入

```
2
S -> a S
S -> #
a
b
```

輸出

```
YES
NO
```

---

04 指定 S 為 start，不使用第一個 LHS：改前 / 改後與測例

第 68 行：用改後取代此行 / 區塊

改前（原碼）

```
        follow[rules[0].lhs].insert("$");
```

改後（對照修改）

```
        follow["S"].insert("$");
```

第 112 行：用改後取代此行 / 區塊

改前（原碼）

```
                words.push_back("$");
std::vector<std::string> stack{"$",
rules[0].lhs}; std::size_t pos = 0, steps = 0;
```

改後（對照修改）

```
                words.push_back("$");
std::vector<std::string> stack{"$", "S"};
std::size_t pos = 0, steps = 0;
```

此變體執行參數：`./main --check`

已試編並執行的變體測例：

輸入

```
2
A -> b
S -> a
a
b
```

輸出

```
ACCEPT
REJECT
```

05 只輸出是否為 LL(1)：改前 / 改後與測例

第 93—103 行：用改後取代此行 / 區塊

改前（原碼）

```
            for (const auto& nt : nonterminals)
{
                std::cout << "FIRST(" << nt <<
")="; print_set(first[nt]); std::cout << "
FOLLOW(" << nt << ")="; print_set(follow[nt]);
std::cout << '\n';
            }
            for (std::size_t i = 0; i <
rules.size(); ++i) { std::cout << "SELECT(" <<
i + 1 << ")="; print_set(select[i]); std::cout
<< '\n'; }
            for (const auto& cell : table) {
                std::cout << "M[" <<
cell.first.first << ',' << cell.first.second <<
"]=";
                for (std::size_t i = 0; i <
cell.second.size(); ++i) std::cout << (i ?
"," : "") << cell.second[i] + 1;
                if (cell.second.size() > 1)
std::cout << " CONFLICT";
                std::cout << '\n';
            }
            std::cout << (ll1 ? "LL1 YES\n" :
"LL1 NO\n");
```

改後（對照修改）

```
std::cout << (ll1 ? "LL1 YES\n" : "LL1 NO\n");
```

已試編並執行的變體測例：

輸入

```
2
S -> a S
S -> #
```

輸出

```
LL1 YES
```

編譯器 Chapter 1 濃縮筆記 — 導論

課程用書：Fischer · Cytron · LeBlanc《Crafting a Compiler》
整理 2026-09-23 | A4 列印版

1 | 編譯器是什麼？

編譯器（compiler）：一支「翻譯程式」的程式。輸入是**原始語言（source language）**寫成的程式文字，輸出是**目標語言（target language）**的程式文字——課程先假設目標是**組合語言（assembly language）**，之後再由**組譯器（assembler）**翻成 CPU 真正能執行的**機器語言（machine language）**。這整個翻譯過程叫 **compilation**。



- **使用者視角**會把上面收成一格：「程式語言進去、機器語言出來」，因為編譯＋組譯通常一個指令做完。但嚴格說編譯器只走到組合語言。
- 編譯器**翻譯程式，不負責執行**。執行是組譯、連結、載入之後的事（或交給直譯器，見 §4）。
- 目標語言不一定是組合語言：也可以是虛擬機指令（§3）或甚至 C（習題 11），都仍算編譯。

2 | 前端（Front-end）與後端（Back-end）



IR（intermediate representation，中介表示）：編譯器內部用來表示程式的資料結構，不是給人看的程式碼。前端把原始碼「分析」成 IR；後端把 IR「合成（synthesis）」成目標碼。

- **前端**依賴來源語言（文法、型別、語意規則）；**後端**依賴目標機器（暫存器、指令集、呼叫慣例）。
- **為什麼要這樣切？**換 CPU 只要重寫後端；換語言只要重寫前端。（補充：M 種語言 × N 種機器，只需寫 M 個前端＋N 個後端，而非 M×N 套完整編譯器。）
- 本課後半的 scanner、parser、type checker、translator 屬前端；optimizer 與 code generator 屬（中）後端。符號表跨兩端（§5）。

易混：這裡的 front / back 跟「使用者介面」無關——front 是**分析端**、back 是**生成端**。

3 | 三種目標碼：編譯器可以產出什麼？

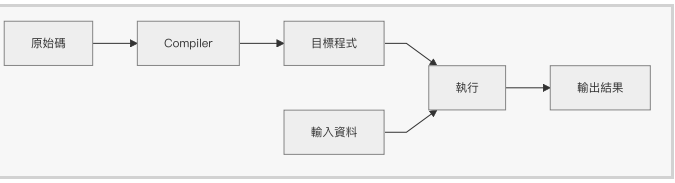
種類	產出內容	用在哪裡
Pure machine code 純機器碼	只有該 CPU 的指令集，**不假設作業系統或函式庫存在**——沒有 read、沒有視窗 API、沒有 malloc，I/O 與記憶體管理全部自己用指令做	實作作業系統、開機載入程式、無 OS 的微控制器（所謂 system implementation languages）
Augmented machine code 擴充機器碼	同一個指令集，但編譯器可以額外發出「呼叫服務」的碼，服務分兩類： ① **OS routines**：system calls、window APIs（作業系統提供） ② **runtime support routines**：語言自帶的執行期支援（啟動碼、例外處理、垃圾回收）	一般使用者空間的 C / C++ 程式——用到 `printf` 就是這類
Virtual machine code 虛擬機碼	全部由**虛擬機（VM）**的指令組成（例：Java 的 **bytecode** 就是 JVM 的指令集）。任何裝了 **VM interpreter** 的電腦都能跑同一份 bytecode	Java 這類重視可攜性的語言

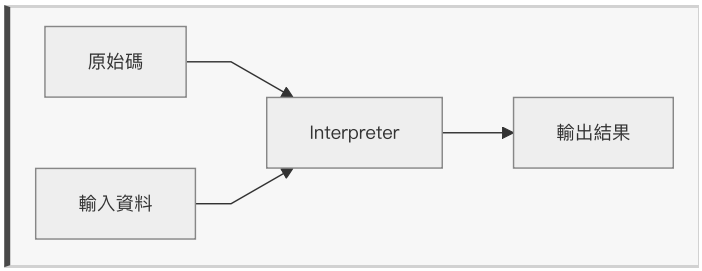
虛擬機的可攜性從哪來？兩句話一起讀：①「只需寫一份 VM interpreter」＝bytecode 格式與編譯器全世界只做一次；②「不同平台需要不同的 VM interpreter」＝每種真實 CPU / OS 還是各要一支能執行 VM 指令的程式。省下的是「每個平台各寫一套完整的 Java→native 後端」。這條路現在划算，是因為 CPU 夠快，多一層解譯的成本負擔得起。

易混：① augmented 不是第三種 CPU，是「同一指令集＋約定好可呼叫的例程」；OS 例程與語言 runtime 是兩類，不要混。② 有了 JVM interpreter **仍然需要 javac**：interpreter 吃的是 bytecode，不是原始碼。

4｜編譯器 vs 直譯器

直譯器（interpreter）：不產生獨立的目標程式，而是**直接執行原始碼**——邊讀程式的輸入資料、邊產出執行結果。它不是「沒有翻譯」，而是把翻譯攤到執行期間做。





	編譯器	直譯器
階段	**compilation**（翻譯）與 **execution**（執行）兩段分開	一段：邊翻譯邊執行
輸入	只有原始碼	原始碼 ＋ 程式的輸入資料
輸出	另一支程式（之後可重複執行）	該次執行的結果
時間觀	編譯一次、執行很多次	每次執行都從原始碼重新走

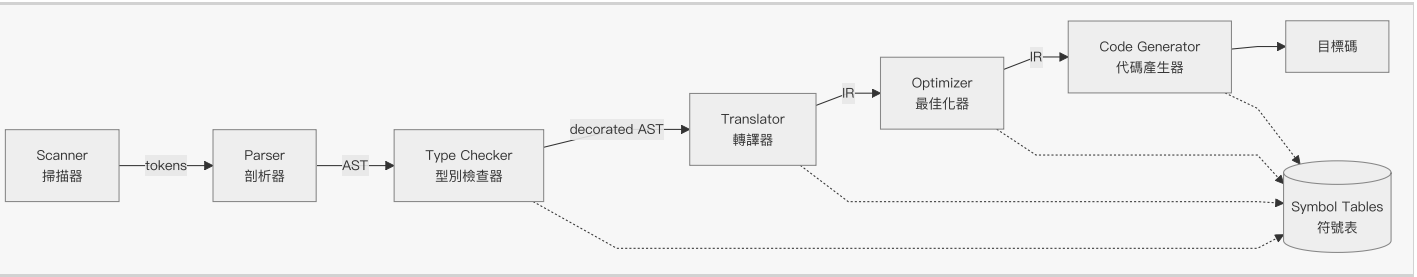
直譯器的三種執行策略

1. **直接執行原始碼**：最單純也最慢。例：LISP、早期 BASIC。
2. **先譯成較有效率的中介表示、立刻執行**：例：Perl、Python、MATLAB、Ruby。
3. **執行「直譯系統內建編譯器」事先編好的碼**：例：UCSD Pascal。

Java 是 2＋3 的混合：`javac` 先把原始碼編成可儲存的 bytecode（像策略 3 的「事先編好」），JVM 執行時拿到的是中介表示而非原始碼（像策略 2）。

> **易混**：語言名稱不能當實作分類——同一個語言可以同時存在編譯實作與直譯實作。

5｜編譯器內部管線：語法導向編譯器（syntax-directed compiler）



實線箭頭＝主鏈資料流（箭頭上是資料單位）；虛線＝對符號表的讀寫。Optimizer 另有「可整個繞過」的捷徑（不最佳化也能生碼）。粗分：Scanner→Translator 是前端（分析），Optimizer、Code Generator 是後端（合成）。

階段	吃	吐	在做什麼＋名詞解釋
Scanner 掃描器	字元	**Tokens**	把字元聚成**詞法單位 token**：`foo_1` 是 1 個 identifier（不是 5 個字元）、`42` 是 1 個 integer。空白通常直接丟掉，不當 token。
Parser 剖析器	Tokens	**AST**	依 **CFG（context-free grammar，上下文無關文法）**檢查結構，通常建出 **AST（abstract syntax tree，抽象語法樹）**——比 parse tree 精簡：丟掉括號、中間非終端等只為語法存在的節點，留下運算與結構。
Type Checker 型別檢查器	AST	**Decorated AST**	又稱 semantic analyzer（語意分析器）。**Decorate（裝飾）**＝在原樹節點上附加型別、常數值、符號表條目等資訊，不是另做一棵樹。檢查型別正確性、reachability（有沒有走不到的碼）、termination、例外處理。
Translator 轉譯器	Decorated AST	**IR**	**語意檢查通過才翻譯**成 IR（例：bytecode、三地址碼）。要求是「意義正確」——IR 的執行效果對應來源語意，不求快。
Optimizer 最佳化器	IR	較好的 IR	改善效能，**語意必須保持不變**；實例見 §7。可以被繞過。
Code Generator 代碼產生器	IR（或直接 AST）	目標碼	把 IR 對應成組合語言（或 VM 指令）。主鏈最後一格。
Symbol Tables 符號表	被右四階段讀寫		記錄每個名稱的型別、位址等資訊。圖上只有 Type Checker、Translator、Optimizer、Code Generator 有虛線連到它；（補充：許多實作在 Scanner 掃到識別字時就會建表，只是圖沒畫。）

> **易混**：parser 接受只代表「合乎 CFG」，不代表型別正確。`int x = "hi";` 可能掃得過、parse 得過，到 type checker 才被擋下。

Scanner 的 C 實作重點（課程補充碼，已驗證可編譯執行）

- `int c = getchar()`：刻意用 `int` 不用 `char`，因為要多留一個負值 `EOF` 表示「讀完」。若用有號 `char`，位元組 `0xFF` 會跟 EOF 撞在一起。
- 送進 `isspace()` / `isalpha()` 前一律先轉 `unsigned char`——負值直接傳入是**未定義行為**（可能查到陣列外面）。
- 詞素放 64 bytes 堆疊陣列，**寫入前**先檢查長度上限，超長就報錯離開，不會緩衝區溢位。
- 「向前看一個字元」靠變數 `c` 保存：每個分支結束時 `c` 已是下一個未處理字元，所以不需要 `ungetc`。
- 實測行為：`foo@` → 印 `IDENT foo` 後報 illegal character；`1foo` → 切成 `INT 1` 和 `IDENT foo`（這是詞法規則的結果，不是 parser 的問題）；連續 64 個字母 → 報 lexeme too long。

6｜Bootstrapping：編譯器編譯自己

> **Bootstrapping（自我編譯）**：當「來源語言＝實作語言」，而且被編譯的原始碼就是這套編譯器本身，這個過程叫 bootstrapping。描述每個編譯器用 **T 圖三欄**：source language（編譯誰） / target language（產出什麼碼） / implementation language（編譯器本身用什麼跑）。

例 A：把 x86 上的 C 編譯器移植到 ARM

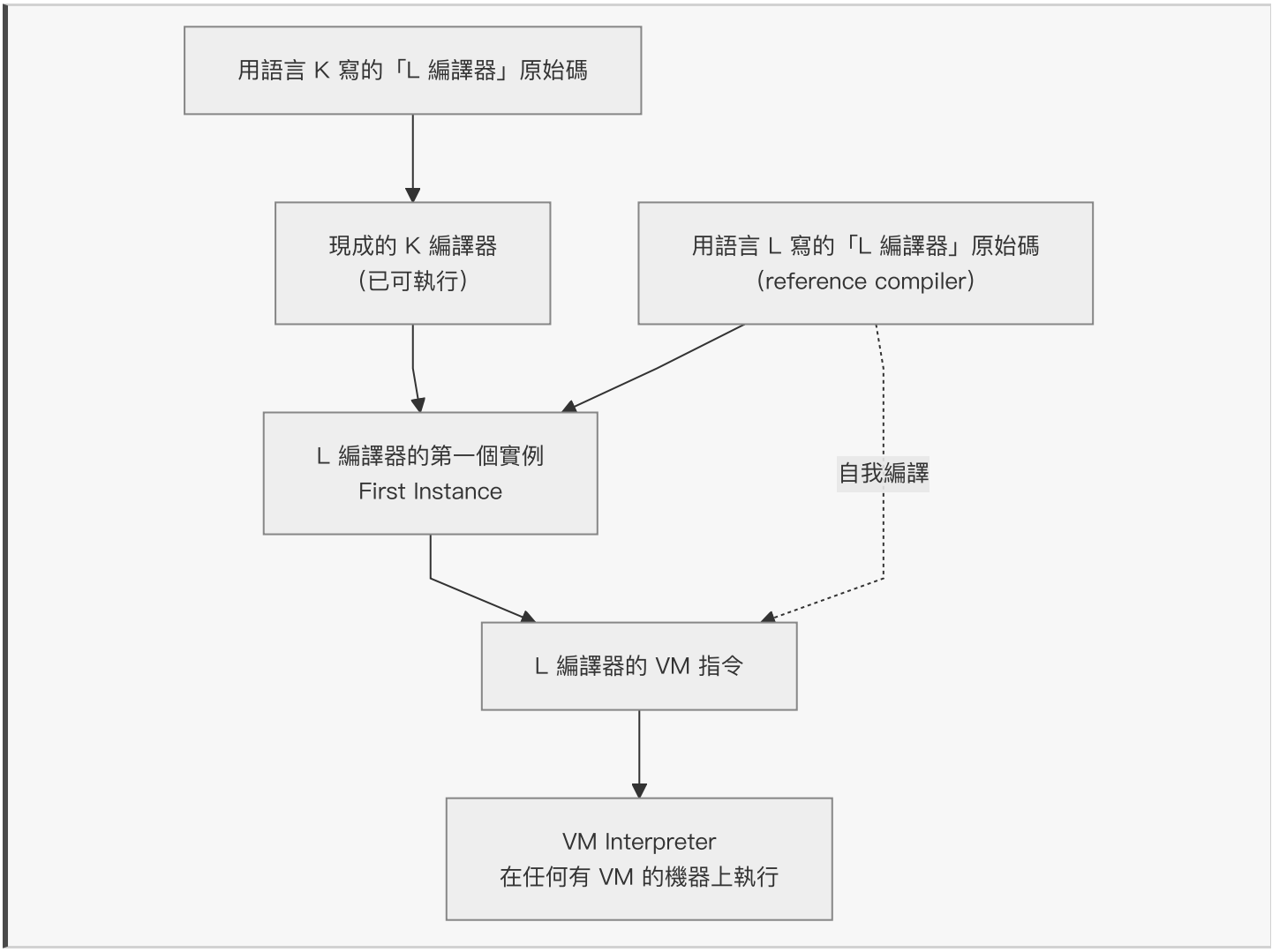
情境：手上有一個跑在 x86 的 C 編譯器 X，它是用 C 寫的（原始碼 P）。ARM 機器上還沒有任何 C 編譯器。做法：把 P 的 code generation 改成產生 ARM 組合語言，再「餵給自己」。

步驟	在哪台機器跑	輸入	產出（source / target / impl）
1. 現況	x86	—（X 已是二元檔）	可跑的 X（C / x86 / C）
2. 改 P 的 code generator	只改原始碼，還沒編譯	P	P′：會發 ARM 碼的編譯器原始碼（C / ARM / C，**還不能跑**）
3. 把 P′ 餵給 X（＝「餵給自己」）	x86	P′	X′：**cross-compiler**（在 x86 上跑、產出 ARM 碼）（C / ARM / **x86**）
4. 再把 P′ 餵給 X′	x86	P′	能在 ARM 上跑的 C 編譯器（C / ARM / **ARM**）

補充 **cross-compiler（交叉編譯器）**：在 A 機器上跑、卻產生 B 機器碼的編譯器。若目標是「ARM 上能跑的編譯器」，只到第 3 步是不夠的——X′ 仍是 x86 二元檔，必須做第 4 步。

> **易混**：只改了 P 的 code generator、還沒重新編譯前，正在跑的 X 發出的仍是 **x86** 碼——跑的是舊二元檔。

例 B：借別的語言跨過第一次（VM 路線）

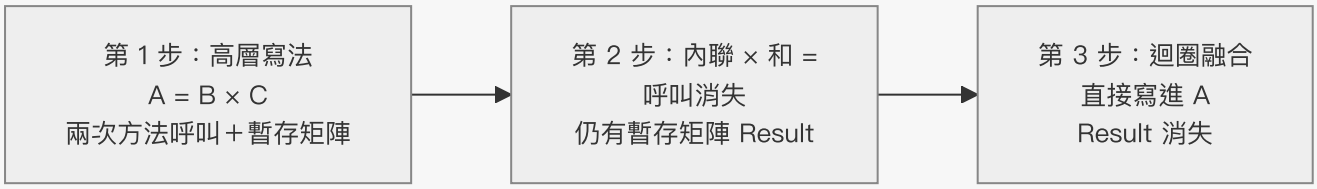


1. 先用**已存在的 K 編譯器**，把「用 K 寫的 L 編譯器」編出來 → 得到 L 的第一個可執行編譯器。這一步跨過「雞生蛋」問題。
2. 再用它編譯「用 L 自己寫的 reference compiler」→ 得到 L 編譯器的 VM 指令。
3. VM 指令交給 VM interpreter 執行——「VM 指令＋interpreter」合起來就是一個**可攜的 L 編譯器**，任何支援該 VM 的架構都能跑，而且它能編譯自己。

K 什麼時候退場？做出第一個實例之後就不需要了；之後只用 L 自己的編譯器與 VM interpreter。這正是「Chicken & Egg」漫畫的答案：第一個編譯器用另一種已存在的語言寫（或先做極小的手寫翻譯器再逐步放大）。

7｜Optimizer 實例：矩陣乘法 A = B × C 的三步改造

名詞先備：**過載運算子（overloaded operator）**＝讓 ×、= 這種符號對自訂型別（矩陣）有意義，底層其實是函式呼叫。**內聯（inlining）**＝把函式本體展開貼到呼叫點，省掉呼叫開銷。**迴圈融合（loop fusion）**＝把兩個跑同樣範圍的迴圈合併成一個。**別名（alias）**＝兩個名字指到同一塊記憶體。



第 1 步：原本的樣子（工作藏在過載運算子裡）

```
A = B × C                      // 高層一行，實際呼叫兩個方法

×(Y, Z)：if Y.cols ≠ Z.rows then 丟例外      ← ① 先查維度
         for i, for j：Result[i,j] ← 0
             for k：Result[i,j] += Y[i,k] × Z[k,j]
         return Result                        // 多一塊暫時矩陣

=(To, From)：if 維度不合 then 丟例外          ← ② 再逐元素拷貝
             for i, for j：To[i,j] ← From[i,j]
```

代價：兩次方法呼叫開銷＋一塊額外的 Result 儲存。

第 2 步：內聯 ①②（呼叫消失）

```
for i = 1..N, for j = 1..N          // 乘法巢
    Result[i,j] ← 0
    for k = 1..N：Result[i,j] += B[i,k] × C[k,j]
for i = 1..N, for j = 1..N          // 拷貝巢（還在！）
    A[i,j] ← Result[i,j]
```

維度檢查在展開圖裡沒畫出來，不代表被刪掉——內聯討論的是熱迴圈。

第 3 步：融合兩個迴圈巢（Result 消失）

```
for i = 1..N
    for j = 1..N
        A[i,j] ← 0
        for k = 1..N
            A[i,j] += B[i,k] × C[k,j]     // 直接寫進 A
```

- 被融合的是「乘法巢」與「拷貝巢」**這兩組**都跑 i、j 的迴圈——**不是**把 i、j 併成單層；i、j、k 三層都還在。
- Result 沒了；**j 迴圈還在**，少掉的是第二組 i、j 拷貝迴圈。

易混：這是編譯器對 IR 做的迴圈融合與暫存矩陣消除，**沒有改矩陣乘法的數學定義**。前提是 A、B、C 互為不同物件；若有別名（例如 `A = A * C`），直接寫 A 可能蓋掉還沒讀完的值。

8｜HW 標準解答（習題 4・8・11）

HW 習題 4：編譯期錯誤 vs 執行期錯誤

> 名詞：**compile-time error（編譯期錯誤）**＝翻譯階段就能發現的錯，整支程式直接不合法；**run-time error（執行期錯誤）**＝實際執行到那一行才爆的錯。對比例子：`a = b/0;`（除數是常數 0，編譯期就看得出）vs `c = 0; a = b/c;`（要到執行才發現）。

題目：該不該讓「含編譯期錯誤的程式」照樣執行、只在真的跑到非法碼時才停？讓程式因為「可能執行到」的非法片段整個不合法，有沒有好處？

> 教材解答 ① Compile-time error is a better approach.
> ② A program with compiler-time errors SHOULD NOT be allowed to execute.
> ③ The advantage is that it ensures all the source codes are syntactically correct.

補充 解答③把好處說成「語法正確」，但除零其實比較接近語意 / 執行期約束，不一定是 CFG 語法錯。另外注意：題目說 `b/0` 會被標成非法是**題目設定**——C 語言裡整數除零是未定義行為，clang 預設只給 warning 照樣產生執行檔；Java 也要加 `-Xlint:divzero` 才警告，執行期才丟 `ArithmeticException`。不要用工具的警告政策改寫教材立場。

HW 習題 8：為什麼做 JVM interpreter 比較容易？

> 背景：C / C++ 通常直接編成真實 CPU 的機器碼（Intel x86、Sparc）；Java 編成 JVM 的機器碼（bytecode），而 JVM 不是真的處理器——它是在既有處理器上被解譯執行的，所以平台獨立。題中的「完整 Java compiler」指「直接對真實 CPU 生碼」的那種，不是 javac。

題目：為什麼為 JVM 做 interpreter 比做完整的 Java compiler 更容易、更快？虛擬機途徑的缺點是什麼？

> 教材解答 Building an interpreter for a virtual machine is easier than building a complete Java compiler is that there are **fewer phases** in developing interpreters. The disadvantages of this virtual machine approach is that the source code must be compiled to byte code, and then a java interpreter can execute it. **This takes more time**.

補充 「少的 phases」對照 §5 管線：完整 native 後端還要做目標指令選擇、暫存器配置、呼叫慣例；VM interpreter 只需實作一組規則的虛擬指令。時間代價＝多一層 bytecode 解譯，跟 §3「CPU 夠快才划算」同一件事。

HW 習題 11：把 C 當「萬用組合語言」

> 名詞：**universal assembly language**＝C 的綽號，因為幾乎每種架構上都有高效率的 C 編譯器。於是有些編譯器乾脆**輸出 C 程式碼**當目標語言，再交給各平台的 C 編譯器走完最後一哩。

題目：編譯器輸出 C（而非特定機器語言）的優點與缺點？

> 教材解答 Generating C code is **more portable** than generating an assembly code for a particular machine language. However, the source code will be translated into C code, and then to assembly code. This is **time-consuming**.

補充 可攜性來自「每台機器早就有 C 編譯器」；代價是多一次 C→assembly 的翻譯，而且 C 的語意（整數寬度、未定義行為、呼叫慣例）會變成你的 IR 必須遵守的邊界。注意：這是把 C 當**目標語言**，跟本課作業「用 C 寫編譯器本身」是兩回事。

9｜自我檢查速答（考前掃一遍）

Q：編譯器輸出 bytecode，還算編譯器嗎？
A：算。廣義上編譯器＝「翻譯到較低階、不再當原始碼改」的翻譯器；目標是 VM 指令或 C 都可以。

Q：使用者視角圖（語言→編譯器→機器碼）看得出前端 / 後端分界嗎？
A：不能，它把編譯器當不透明方塊，還順手合併了組譯器。

Q：只換 CPU、語言不變，要重寫哪一端？
A：後端（code generator＋部分 runtime）；前端與 IR 留用。

Q：管線圖上哪幾個方塊有虛線連到符號表？
A：Type Checker、Translator、Optimizer、Code Generator 四個；Scanner、Parser 沒有。

Q：編譯器的「執行階段」跑的是 source 還是 target？
A：target。source 只在 compilation phase 出現。

Q：`foo1` 是幾個 token？`1foo` 呢？
A：`foo1` ＝1 個 identifier；`1foo` ＝ `INT 1` ＋ `IDENT foo` 兩個 token（詞法規則問題）。

Q：迴圈融合後還有 Result 嗎？還有 j 迴圈嗎？
A：Result 沒了；j 還在，少掉的是第二組 i、j 拷貝迴圈。

Q：VM 路線 bootstrapping 中，哪一步之後就不再需要語言 K 的編譯器？
A：做出「L 的第一個實例」之後；之後只用 L 自己的編譯器與 VM interpreter。

10｜名詞速查表

名詞	一句話解釋	名詞	一句話解釋
compiler 編譯器	把程式從來源語言翻成目標語言的程式	interpreter 直譯器	不產出目標程式，直接執行原始碼
assembler 組譯器	把組合語言翻成機器碼	compilation	「原始碼→目標碼」的翻譯過程
front-end 前端	分析原始碼、建出 IR 的那半	back-end 後端	把 IR 合成成目標碼的那半
IR 中介表示	編譯器內部的程式表示，不給人看	symbol table 符號表	記錄名稱的型別、位址等資訊的表
token	詞法單位（identifier、integer…）	CFG	上下文無關文法，描述語法結構
AST 抽象語法樹	去掉括號等語法雜物後的程式樹	decorated AST	節點附上型別等語意資訊的 AST
pure machine code	不假設 OS / 函式庫存在的純機器碼	augmented machine code	機器碼＋約定可呼叫的 OS / runtime 例程
VM / bytecode	虛擬機 / 它的指令集（如 JVM bytecode）	runtime support	語言自帶的執行期支援（啟動碼、GC…）
bootstrapping	編譯器編譯「自己」的原始碼	cross-compiler	在 A 機跑、產 B 機碼的編譯器
inlining 內聯	把函式本體展開到呼叫點	loop fusion 迴圈融合	合併範圍相同的迴圈，省暫存
compile-time error	編譯期即可發現的錯誤	run-time error	執行到該行才出現的錯誤
undefined behavior	語言不定義後果的行為（如 C 除零）	universal assembly language	C 的綽號：到處都有它的編譯器

Chapter 2 | A Simple Compiler

考場找答案：token / RE → 本頁；CFG / predict / parser → p.2；推導 / parse tree / AST → p.3；symbol table / type / error → p.4；dc code → p.5；課本習題 / scanner 擴充 → p.6。
題型依據：第一次小考的 Explain / Difference / 情境判斷。以下英文問答為**仿題型練習答案**，不是教師公布的第二次考題。

1 | ac 編譯器：每一站在處理什麼？ PPT 2、12、15、27–38

階段	輸入 → 輸出	關鍵工作
Scanner / lexer	Characters → tokens	依 regular expression 辨認詞法單位，略過 blank。
Parser	Tokens → AST	依 CFG 檢查語法；本章用預測式 recursive descent。
Symbol-table constructor	AST 宣告 → symbol table	記錄 identifier 的型別，檢查重複宣告。
Type checker	AST + symbol table → decorated AST	查宣告、由葉往根求型別，必要時加入 int2float。
Code generator	Checked AST → dc code	走訪節點；先產生運算元，再產生運算子。

短答：The scanner groups characters into tokens. The parser checks their grammatical structure and builds an AST. Semantic analysis checks declarations and types; code generation translates the checked AST into dc instructions.

2 | Token：kind 與實際字串 PPT 3–5、12–14

Explain what a token / lexeme is.

A token is a lexical unit with a type and, when needed, a value. Its lexeme is the actual character sequence in the source.

`b`：type = `id`，value = `"b"`。`3.2`：type = `fnum`，value = `"3.2"`。Parser 選規則看 **type**，符號表 / 產碼需要 **value**。

Token	ac 規則 / lexeme
floatdcl / intdcl / print	`f / i / p`
id	`[a-eghj-oq-z]`，單一字母
assign / plus / minus	`= / + / -`
inum	`[0-9]+`
fnum	`[0-9]+\.[0-9]+`
blank	原表為一個以上空格；略過
$	輸入結尾；不是要打的字元

RE 的 `+` 是一次以上；`\.` 是字面小數點。原圖未跳脫，此處消除歧義。多行程式另需略過換行。

ac ≠ C：id 只有一個小寫字母，f、i、p 是保留字；沒有分號、括號、乘除或 `//` 註解。投影片的註解是給人看的。

3 | 切 token：短例快查

輸入	Scanner 的結果
`b2`	id(b), inum(2)；不是單一 id
`abc`	id(a), id(b), id(c)
`007`	一個 inum(007)
`-5`	minus, inum(5)；負號非數字一部分
`.5`	首字元「.」無 token → lexical error
`5.`	Fig 2.3 不合法；Fig 2.6 虛擬碼會收

4 | RE 與 CFG 的分工

What is the difference between a regular expression and a context-free grammar?

In this compiler, regular expressions specify the character patterns of tokens. A context-free grammar specifies how tokens form declarations, expressions, statements, and programs.

terminal：不能再改寫，本章是 token kinds。
non-terminal：可依 production 改寫。
production：`A → α`，將 LHS 的 A 換成 RHS 的 α。
start symbol：本章為 Prog。
λ / ε：空字串，不消耗任何 token。
$：end-of-input marker，與 λ 不同。

Why are scanning and parsing separate?

Scanning handles character-level patterns; parsing handles token-level structure. This separation simplifies the grammar and lets the parser operate on token types.

5 | 課本範例：一路都用它 PPT 5

```
f b
i a
a = 5
b = a + 3.2
p b
```

```
floatdcl id | intdcl id | id assign inum
id assign id plus fnum | print id | $
```

共 14 個來源 tokens（| 僅分句）；再加 scanner 回傳的 `$` 共 15 個。空白不計。宣告 b:float、a:integer；最後印出 8.2。

6 | Scanner 作答骨架

略過 blank → EOF 回 $ → 數字呼叫 SCANDIGITS → 否則吃一字元分類 → 未定義字元報 lexical error。

PEEK：看但不吃；**ADVANCE**：吃掉並前進。Scanner 的 lookahead 是**字元**；parser 的 lookahead 是 **token**。

5. 的答法：問 formal definition → 不合法；問「照 Fig 2.6 執行」→ fnum，因小數點後的 while 可跑 0 次。

文法 × Predict Set × Parser

7｜文法與每條產生式的 predict set

PPT 7、17–24；全表依文法推算

#	Production	Predict set	選它時做什麼
1	`Prog → Dcls Stmts $`	{floatdcl, intdcl, id, print, $}	DCLS(); STMTS(); MATCH($)
2	`Dcls → Dcl Dcls`	{floatdcl, intdcl}	剖析一個宣告，再繼續
3	`Dcls → λ`	{id, print, $}	宣告結束，不吃 token
4	`Dcl → floatdcl id`	{floatdcl}	MATCH(floatdcl); MATCH(id)
5	`Dcl → intdcl id`	{intdcl}	MATCH(intdcl); MATCH(id)
6	`Stmts → Stmt Stmts`	{id, print}	剖析一個敘述，再繼續
7	`Stmts → λ`	{$}	敘述結束，不吃 token
8	`Stmt → id assign Val Expr`	{id}	指定敘述
9	`Stmt → print id`	{print}	只能印變數，不能 p 5 或 p a+1
10	`Expr → plus Val Expr`	{plus}	繼續處理「+ 值」
11	`Expr → minus Val Expr`	{minus}	繼續處理「− 值」
12	`Expr → λ`	{id, print, $}	算式結束；lookahead 留給下一句
13	`Val → id`	{id}	變數
14	`Val → inum`	{inum}	整數常數
15	`Val → fnum`	{fnum}	浮點常數

7 個 non-terminals：Prog, Dcls, Dcl, Stmts, Stmt, Expr, Val。**10 個 terminals**：floatdcl, intdcl, print, id, assign, plus, minus, inum, fnum, $。Blank 已略過，λ 不是 terminal。

> **文法限制**：宣告在全部敘述前；Val 不接受負號 / 括號。`a=5` 是 **Val(5) + Expr(λ)**，Expr 只是「後續 ± Val 串」。Dcls、Stmts 都可為空，所以空輸入在此 CFG 中合法。

8｜Predict 怎麼算、怎麼解釋？

What is a predict set? Why must the sets be disjoint?

A predict set contains the lookahead tokens for selecting a production. For predictive parsing with one token of lookahead, alternatives of the same nonterminal must have disjoint predict sets so that the choice is unique.

1. RHS 第一個是 terminal t → {t}。
2. 第一個是 non-terminal → 找它能導出的第一個 token；若可為空，繼續看後面符號。
3. 整個 RHS 可為 λ → 加入 LHS 後面能接的 token。

> **補充公式（FIRST / FOLLOW）：**
> Predict(A → α) = FIRST(α) − {λ}；
> 若 α ⇒* λ，再聯集 FOLLOW(A)。
> 這是推算工具，Ch2 先用口語說明。

Stmts → Stmt Stmts：Stmt 可由 id 或 print 開始，故 {id, print}。
Stmts → λ：Prog 裡 Stmts 後面是 $，故 {$}。
Expr → λ：後面是下一個 Stmt 或程式結尾，故 {id, print, $}。

> λ 分支**不能**寫「前面分支都不是就 return」。Expr 遇到 inum 並不合法；λ 不表示吞掉未知輸入。

9｜Recursive descent 的程序模板

Recursive descent uses one procedure per nonterminal. It selects a production using lookahead, matches terminals, and calls procedures for nonterminals in RHS order.

```
教材虛擬碼整理（不建 AST 的辨識版）
STMT():
  if peek() == id:
    match(id); match(assign)
    VAL(); EXPR()
  else if peek() == print:
    match(print); match(id)
  else: error()

STMTS():
  if peek() in {id, print}:
    STMT(); STMTS()
  else if peek() == $: return   // λ
  else: error()
```

MATCH(t)：確認 lookahead 為 t，成功才消耗；不符即 syntax error。**LL(1)**：由左到右讀輸入（L），最左推導（L），看 1 個 token。

What if one lookahead cannot select a unique rule?

Rewrite the grammar or use more lookahead, if sufficient. Overlapping predict sets prevent a unique LL(1) choice; they do not by themselves prove ambiguity.

Recursive descent 也可能使用回溯；本章採 predict set 選分支。不能推論所有文法都可藉增加 lookahead 解決。

Derivation × Parse Tree × AST

10｜三種表示，回答的問題不同

PPT 8–11、27–28

	Derivation 推導	Parse tree 剖析樹	AST 抽象語法樹
重點	依序用了哪些 productions	文法規則如何形成階層	程式操作與運算元
保留	每一步 sentential form	non–terminals、terminals、λ、$	宣告、assign、plus/minus、print、值
省略	—	獨立子樹的改寫先後	Dcls/Stmts/Val/Expr 包裝、λ、$

Difference：A parse tree records the full grammatical structure of a derivation. An AST omits grammar–only details and keeps the operations and operands needed by later compiler phases.

最左推導：每次展開最左 non–terminal；**最右推導：**每次展開最右 non–terminal。同一棵 parse tree 的最左與最右推導只差展開順序，不因此變成兩棵不同的樹。

11｜完整最左推導：i a a=5 p a

```
Prog
⇒ Dcls Stmts $                          (1)
⇒ Dcl Dcls Stmts $                      (2)
⇒ intdcl id Dcls Stmts $                (5)
⇒ intdcl id Stmts $                     (3)
⇒ intdcl id Stmt Stmts $                (6)
⇒ intdcl id id assign Val Expr Stmts $ (8)
⇒ intdcl id id assign inum Expr Stmts $ (14)
⇒ intdcl id id assign inum Stmts $    (12)
⇒ intdcl id id assign inum Stmt Stmts $ (6)
⇒ intdcl id id assign inum print id Stmts $ (9)
⇒ intdcl id id assign inum print id $  (7)
```

共 11 次改寫，12 個句型。規則序列：`1,2,5,3,6,8,14,12,6,9,7`。

最後是 token kinds；a、5 是 attributes。考題要「程式字串」時，可在 token 葉旁註記 lexeme。

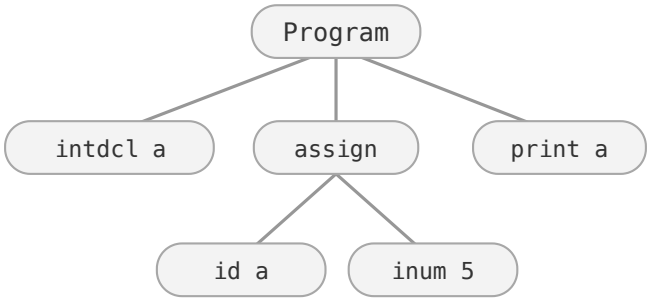
12｜畫 parse tree 的檢查法

1. 根為 Prog，孩子 Dcls、Stmts、$。
2. 每次展開完整照 RHS 左到右放孩子。
3. Dcls / Stmts 遞迴串接，最後 λ 收尾。
4. 指定式：id、assign、Val、Expr。
5. 葉由左到右忽略 λ，等於完整 token 串。

不要提前變成 AST：parse tree 的 plus 是 Expr 底下的 terminal 葉；AST 的 plus 才是兩個運算元的父節點。嚴格文法樹中 Val 下保留 inum / id 這層。

13｜同一例的 parse tree 與 AST

```
Parse tree（括號為 lexeme 註記）
Prog
├─ Dcls
│  ├─ Dcl
│  │  ├─ intdcl (i)
│  │  └─ id (a)
│  └─ Dcls → λ
├─ Stmts
│  ├─ Stmt
│  │  ├─ id (a)
│  │  ├─ assign (=)
│  │  ├─ Val → inum (5)
│  │  └─ Expr → λ
│  └─ Stmts
│     ├─ Stmt
│     │  ├─ print (p)
│     │  └─ id (a)
│     └─ Stmts → λ
└─ $
```



AST 重新繪製，保留語意結構；不是投影片原圖。

14｜遞迴方向不等於運算結合性

PPT 28、42；補充辨析

本課習題圖： `x+30+1` 畫成 `plus(x, plus(30,1))`（右側較深）。要求依教材圖畫，就照該形狀。
一般左結合語意 / 本地 C 實作： `minus(minus(x,30),1)` 表示 `(x–30)–1`。x=100 時得 69；`x–(30–1)` 得 71。未指定時說明採用的結合規則，不能只憑右遞迴斷定語意。

A right–recursive grammar does not force a left–associative language to use a right–associated AST. Semantic actions can accumulate the left operand to construct a left–associated tree.

本章整數加法例兩種形狀都得 131；不能泛化成任何數值語意下都可任意重排加法。

Symbol Table × Type × Errors

15 | Symbol table

PPT 29–33

What does the symbol table store? Why 23 entries?

The ac symbol table maps each identifier to integer, float, or unused (null). There are 23 possible identifiers: the 26 lowercase letters excluding the reserved letters f, i, and p.

動作	規則
初始化	全部設 null（尚未宣告）
EnterSymbol(name,type)	null 才填入；已有型別 → duplicate declaration
LookupSymbol(name)	回傳型別；null 代表未宣告
範例 f b / i a	b→float、a→integer；其他 21 項 null

> null 是 unused 狀態，**不是第三種運算型別**。表記錄型別，不是 a=5、b=8.2 這些執行期值。C 補充實作用 26 格陣列方便索引，課本仍是 23 個可用名稱。

16 | Type checking 為何 bottom-up？

The type of a compound expression depends on its operands. Therefore, type checking processes the children before the parent, looking up identifier types and inserting conversions when needed.

節點	規則 / 依據
inum / fnum	integer / float（p.034）
id	查表；須已宣告（p.033）
plus / minus	依子節點型別決定自己型別
assign	RHS 與目標 id 型別相容
int2float	編譯器加入（Fig 2.13）

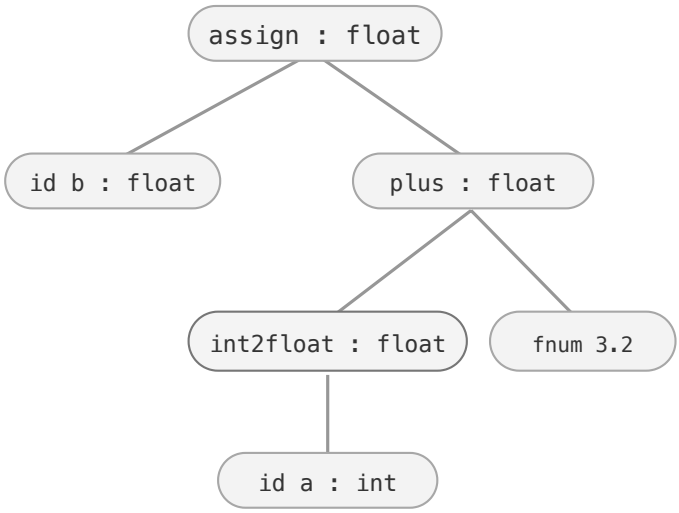
> **來源界限**：p.034 的 utility methods（Fig 2.12）未附。Fig 2.13 確實把 `a+3.2` 的 a 包成 int2float。下面完整規則依此反推、由本地實作採用，**不是逐字教材規格**。

補充規則	結果
int ± int	int
int ± float / float ± int	int 那側轉 float，結果 float
float ± float	float
float ← int	RHS 外包 int2float
int ← float	拒絕隱式窄化（本地實作）

17 | Decorated AST：轉換插在哪？

PPT 35

`f b i a a=5 b=a+3.2 p b` 中第二個指定：



保留 a 的宣告：a 仍是 integer，只在這次運算使用其值時轉成 float。不是把 symbol table 的 a 改成 float。

A decorated AST carries semantic information such as types and may contain inserted conversion nodes. Here, int2float converts the value of a before floating-point addition.

18 | Semantic action：執行或記錄？

PPT 25–27

What happens after parsing a = 5?

An interpreter may execute the assignment immediately. In this chapter's compiler, the semantic action records the assignment in an AST for later analysis and code generation.

走完 `id assign Val Expr` 已知道目標和 RHS，可建 assign 節點。**本章**先建 AST，再檢查、產碼；不是所有 compiler 都必須如此排程。

19 | 情境題：Lexical / Syntax / Semantic？

依 ac 規則自編練習

輸入（各列獨立）	第一個問題	可直接寫的英文理由
`i a a=@`	Lexical	The character @ does not match any ac token.
`i a a=+5`	Syntax	Val cannot begin with plus; ac has no unary plus here.
`i a p 5`	Syntax	The print production requires an identifier, not a number.
`i a a=5 f b`	Syntax	Declarations cannot occur after statements.
`i a f a`	Semantic	The identifier a is declared twice.
`f b b=c`	Semantic	The identifier c has not been declared.
`i a a=3.2`	Semantic（補充型別規則）	A float value cannot be implicitly assigned to an integer variable.

> **通用答法**：A syntax error violates the grammar. A static semantic error violates declaration or type rules even though the token sequence is grammatically valid.
> 合法程式算出非預期答案可能是 logic error；編譯器不一定知道人的意圖。未宣告與未初始化也是兩個不同問題。

Code Generation | dc 堆疊機

20 | 六種 visitor：寫出碼的模板

PPT 36–38 · Fig 2.14–2.15

AST 節點	依序輸出的碼	意義
Consting：inum / fnum	`value`	常數推入堆疊
SymReferencing：id r	`lr`	複製暫存器 r 的值到堆疊
Computing：L op R	`code(L); code(R); op`	先左、再右、最後運算子
Converting：int2float(E)	`code(E); 5 k`	教材以設定 scale 表達轉換
Assigning：r = E	`code(E); sr; 0 k`	彈出 RHS 結果存入 r，重設 scale
Printing：p r	`lr; p; si`	取值、印值、彈出丟到暫存器 i
宣告	無	只影響符號表，不輸出 dc 指令

分號僅分隔「依序輸出」的指令，不是 dc 字元。`lr/sr` 的 r 是變數名，例如 `la/sa`。

> **Why postfix?** A stack machine needs the operands before the operator. The code generator therefore emits code for the left child, then the right child, and finally the operation.
> 走訪從根呼叫，不代表先輸出根的運算子；計算節點實際 emit 順序是後序（postorder）。

21 | 完整例：b = a + 3.2

```
ac source       dc target
f b             (宣告無碼)
i a             (宣告無碼)
a = 5           5
                sa
                0 k
b = a + 3.2     la
                5 k
                3.2
                +
                sb
                0 k
p b             lb
                p
                si
```

轉換位置： `int2float(id a)` → 先 `la` 再 `5 k`。不是在整個加法後才放轉換。

22 | dc 指令的易混處

- `sr` 會 **pop**；`lr` 只讀暫存器、push 副本。
- `p` 印出 top，**不 pop**；用 `si` 清理。
- i 是 ac 保留字、不能當變數名，作 dc 垃圾暫存器不會覆蓋使用者變數。
- `5 k`：push 5，再由 k pop 5 作 scale；不是把原有 top 改成 5。
- `0 k` 不把已存入暫存器的 8.2 變成 8。

> **教材模型與 dc 行為：** `k` 設定 scale，並非硬體的 int→float 指令。dc 的加減小數位由運算元決定，不能說 `5 k` 使所有輸出固定五位；考產碼仍照 Fig 2.14。

23 | 手算 stack：右側是頂端

指令	執行後 stack	其他狀態
`5`	[5]	—
`sa`	[]	a = 5
`0 k`	[]	scale = 0
`la`	[5]	a 保持 5
`5 k`	[5]	scale = 5
`3.2`	[5, 3.2]	—
`+`	[8.2]	5 + 3.2
`sb`	[]	b = 8.2
`0 k`	[]	scale = 0
`lb`	[8.2]	—
`p`	[8.2]	印出 8.2
`si`	[]	i = 8.2（丟棄用）

24 | 減法：一定要分左右

`L R -` 計算 **L − R**；先 pop 的是右運算元 R，第二個 pop 才是 L。

```
左結合：(100 - 30) - 1
100 30 - 1 -        → 69

右結合：100 - (30 - 1)
100 30 1 - -        → 71
```

`x=x-30` 產碼：`lx 30 - sx 0 k`。不是 `30 lx -`，也不應先 `sx` 才計算 RHS。

Why is si emitted after p?

The dc print command leaves the value on the stack. The generated si pops it into the unused register i, restoring the stack for subsequent statements.

教材習題 × Scanner 擴充

25 | HW 1–2 PPT 39–44

題目： `i x x=100 x=x+30+1 p x`

HW 1：parse tree / 最左推導

按 p.2 文法，規則序列為：

`1,2,5,3,6,8,14,12,6,8,13,10,14,10,14,12,6,9,7`（19 次改寫）。

第二句指定的尾巴：

`Expr → plus Val Expr`，Val→inum(30)；下一層 `Expr → plus Val Expr`，Val→inum(1)；最後 Expr→λ。

HW 2(a)：教材 AST 的結構

```
Prog
├─ intdcl x       // 原圖只標 x
├─ assign(id x, inum 100)
├─ assign
│  ├─ id x
│  └─ plus
│     ├─ id x
│     └─ plus
│        ├─ inum 30
│        └─ inum 1
└─ print x
```

補齊節點種類；樹形保留 p.042 的右結合。

HW 2(b)：語意正確嗎？

> **教材解答 p.043：**先答「No」，接著說 AST 的 30、+、1 可合成單一節點 31。
> **待核對：**p.033–034 所列宣告 / 型別規則未顯示此例違規。把常數子樹算成 31 是 **constant folding（常數摺疊）**，不等於修正語意錯誤。問「依投影片解答」時保留 No 及說明；問規則推理時明列 x 已宣告、運算元皆 integer。不可捏造「ac 不准連續加法」。

HW 2(c)：教材摺疊後的 dc code

```
100 sx 0 k
lx 31 + sx 0 k
lx p si
```

印出 **131**。原右結合 AST 未摺疊的第二行：`lx 30 1 + + sx 0 k`。本地 C 左結合版：`lx 30 + 1 + sx 0 k`。本例結果相同，形狀與碼序不同。

26 | HW 3：擴充 scanner PPT 45–46

(a)(b) 宣告可用完整單字

```
floatdcl  "f" | "float"
intdcl    "i" | "int"
```

程式也要改：看到 f/i 時不能立刻只回一個字元；檢查完整關鍵字是否匹配，未匹配字元保留給後續 token。明定最長匹配 / 關鍵字邊界規則。

原 ac 的 `fl` 是 `floatdcl id(l)`，仍須辨認。完整 `float` 匹配時選長 token，不能拆成 f、l、o、a、t。

(c) 可選的有號指數

教材原解答（照錄結構）：

```
inum  [0-9]+ e -? [0-9]+
fnum  [0-9] . [0-9]+ e -? [0-9]+
```

> **與題意有三個落差：**① 指數變必填；② 只收負號、漏正號；③ fnum 小數點前只有一位。「依題意擴充」時應補齊。

依題意補充：接受原數字及指數形式；帶指數者歸 fnum（`123e-22` 不一定是整數）。

```
inum  [0-9]+
fnum  [0-9]+\.[0-9]+(e[+-]?[0-9]+)?
      | [0-9]+e[+-]?[0-9]+
```

`e` 依題目小寫；要 E 另明定 `[eE]`。流程：數字 → 可選小數部分 → 可選 e → 可選 +/− → **至少一位指數數字**。

應接受（補充規則）	非完整數字格式
5、3.2、1.0e10 123e-22、123.4e+5	1e、1e+、3.e2、.5

不完整指數依明定 scanner 規則報 malformed number，或退回較短 token 由後續階段拒絕；RE 本身未決定所有錯誤回復策略。

Extension answer：Update both the token patterns and the scanner. Recognize the optional exponent and its sign, require exponent digits, and preserve characters that belong to the next token.

後端：dc 不直接接受科學記號；須轉成可輸出的十進位寫法，不能直接 EMIT `1e10`。

27 | 最後 30 秒：英文短答骨架

> **Explain X：**definition → role / input–output → tiny example。
> **Difference：**“X checks / represents …, whereas Y checks / represents ….”
> **Is it valid?：**“Lexically …; syntactically …; semantically …, because ….”
> **How to extend?：**token rule → scanner behavior → parser/type/codegen 是否受影響。

來源與範圍：Ch02 a simple compiler.pptx / 46 頁衍生 PDF、四份逐頁伴讀（p.001–046）；題型參考「Compiler – 第一次小考題目與標準答案」，版面參考 Ch1 cheatsheet。6 頁 A4（雙面 3 張），未假設允許的頁數上限。
參照：Chapter 2 原投影片 PDF · 第一次小考 · GNU dc manual。
待核對保留：p.034 缺 Fig 2.12；p.043 的 No 與既列規則不一致。p.014、p.042、p.046 的定義 / 樹形 / 解答差異已明示。

本單元名詞｜考場直接查

Explain / Define：寫中欄英文；例子看右欄。**How：**照 p.2 流程。**Why CFG / AST：**詳解 p.3 / p.4。本頁為 Ch3 術語與前兩次小考補充。

A｜字元、token 與文法

Ch3 PPT 2–5、31、51；Ch2＋補充

名詞 / notation	English definition（可作短答）	中文辨析 / 例子
Scanner / lexer	Groups source characters into tokens.	掃描器：字元流 → token 流。
Token / token type	A lexical unit / its category used by the parser.	type 如 ID、NUM；不是字元數。
Lexeme	The actual source character sequence matched.	詞素：ID 的 lexeme 可為 count。
Pattern / regular expression	Pattern: valid lexemes. RE: a description of a regular language.	RE 是規格；lexeme 是實例。
Attribute / transducer	Attribute: token data. Transducer: a recognizer that also produces output.	如 007 的數值 7；不只判合法。
Parser / CFG	Parser: checks token structure. CFG: each rule has one nonterminal on its left.	CFG 左側是一個 nonterminal。
Terminal / nonterminal	Terminals are not rewritten; nonterminals are expanded by productions.	parser 文法的 terminal 通常是 token type。
Production / start symbol	A rewrite rule / the symbol from which derivation begins.	如 S→aSb；從 S 開始推導。
AST / parse tree	An AST keeps essential structure; a parse tree records grammar expansions.	AST 留運算 / 敘述，省語法包裝。

B｜語言集合與 RE operators

PPT 5–16

名詞 / notation	English definition（可作短答）	中文辨析 / 例子
Alphabet Σ / string / language	A finite symbol set / a finite sequence / a set of strings.	符號集合 / 一串符號 / 字串集合。
Empty string λ or ε / ∅	λ is a zero-length string; ∅ contains no strings.	{λ}≠∅；λ 不消耗輸入。
Regular language	A language recognizable by a finite automaton.	RE、NFA、DFA 表達同類語言。
Literal / metacharacter	A literal denotes itself; a metacharacter controls the pattern.	字面 a；* 是 RE 操作符。
Alternation A | B	The union of the two languages.	選 A 或 B；不是串接。
Concatenation AB	Strings from A followed by strings from B.	順序重要；通常 AB≠BA。
Kleene A* / positive A+	Zero-or-more / one-or-more concatenated repetitions.	* 一定含 λ；A 含 λ 時 + 也可含 λ。
Optional A? / A^k	Zero or one occurrence / exactly k concatenated copies.	A?=A|λ；A^0=λ。
Character class / Not(C)	A set of single characters / its complement within Σ.	Not(C)=Σ\C；非任意語言補集。

C｜Finite automata 與子集構造

PPT 23–50

名詞 / notation	English definition（可作短答）	中文辨析 / 例子
DFA / NFA	DFA: one successor per symbol. NFA: multiple choices or epsilon moves.	能力相同；NFA 接受看存在路徑。
State Q / q_0 / F	Q is the finite state set; q_0 starts; F accepts.	M=(Q,Σ,δ,q_0,F)；雙圈表示 F。
Transition δ / Δ	A DFA successor / a set of NFA successors.	轉移表：row=state，col=輸入。
Epsilon transition	A move that consumes no input character.	λ 邊不代表讀一個 λ 字元。
λ-closure close(S)	States reachable from S by zero or more epsilon moves.	包含 S 本身；沿 λ 走到固定點。
move(S,c)	All successors after consuming one c from states in S.	先聯集，之後才做 close。
Subset construction	Uses sets of NFA states as DFA states.	集合 S 含任一接受 state 即接受。
Dead state / sink	Dead: no acceptance is reachable. Sink: every input loops there.	本課 X=∅ 是拒絕 sink。
Worklist / seen	Unprocessed discovered subsets / all discovered subsets.	新集合只入隊一次；按內容去重。

D｜掃描行為與錯誤

教材整理＋補充辨析

名詞 / notation	English definition（可作短答）	中文辨析 / 例子
Maximal munch / tie priority	Take the longest match; flex breaks equal-length ties by rule order.	floatx 選整個 ID；同長才比先後。
Lookahead / backtracking	Inspect upcoming input / return to a saved input position.	PEEK 不吃；ADVANCE 吃；記最後接受。
Recognize / tokenize	Test a whole string / split input into successive tokens.	完整 NUM 拒 12e；仍可能切 NUM+ID。
EOF / EOL	End of input / end of line.	檔尾≠換行；教材 // 註解要求 EOL。
Semantic action / semantic analysis	An action attached to a rule / checks of declarations and types.	本課 action 建 AST；型別另檢查。

三個判定句：Lexical error: no valid token match. Syntax error: the token sequence violates the grammar. Static semantic error: valid syntax violates declaration or type rules.　**注意：**接受整串必須讀完；scanner 切 token 則取合法前綴。

常見流程｜看到題目就照步驟做

直接定位：Define / Difference → p.1；scanner 如何建立 → F1；切 token → F2；RE→NFA → F3；NFA→DFA → F4；畫 AST → F5；判 error / Why CFG → F6。本頁為課程流程摘要與補充微型例題。

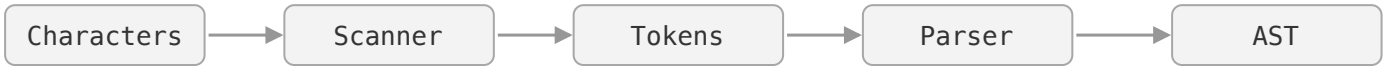
F1｜建 scanner 與跑 compiler：兩條流程

PPT 2、33；Ch2＋補充串連

建立時：用 token patterns 建 NFA，子集構造成 DFA，再輸出表或控制程式。



執行時：scanner 依 pattern 切字元；parser 依 CFG 組 token，建 AST。



後續：AST ＋ symbol table → 檢查宣告 / 型別 → decorated AST → 產碼。圖為本課主線摘要；**RE→NFA 是建立工具，characters→tokens 是處理來源程式。**

F2｜給短字串，列 tokens

詳 p.6

① 看題目 token 規格 / 語言。② 從當前位置找最長合法前綴。③ flex 同長取先列 rule。④ 回 type，附必要 attribute；略過空白 / 註解。⑤ 移到下一 token，直到 EOF。

> **例：**keyword=float，ID=本章 identifier：
> `float x=12;` → FLOAT、ID(x)、ASSIGN、NUM(12)、SEMICOLON。`floatx` → 一個 ID。

長候選失敗：回最後接受位置；沒有接受前綴才無 token 可回。Choose the longest valid prefix; resolve equal-length matches by rule priority.

F3｜給 RE，畫 NFA

模板 / 完整例 p.7

① 先拆 (…)、* / +、串接、|。② literal 畫一條字元邊。③ | 加 λ 分支；AB 接 A 出口到 B 入口。④ * 加零次旁路及重複邊；+ 去掉外部零次旁路。⑤ 標整體 start、F、Σ。

> `(bc)*d`：先 b 串 c → 對 bc 加 * → 再串 d。測 **d ✓、bcd ✓、λ ✗、bc ✗**。

交卷檢查：所有邊有 label；只把完整表達式出口標接受。若內層 A 含 λ，A+ 仍可收 λ。

F5｜給短程式，畫 AST

4 棵完整樹 p.4

① 確認語言及結合性。② 按來源順序列敘述。③ 指定句先畫 assign（目標，RHS）。④ RHS 按括號 / 優先序 / 結合性遞迴拆。⑤ 不畫純語法包裝，保留識別字與常數。

> 一般 C-like `x=a+b*c;`：
> `assign(x, plus(a, times(b,c)))`。
> **整句根=assign，RHS 根=plus**，不可把 + 畫在 * 的下面。

以還原括號式驗算。題目要 AST，就畫節點與邊；未要求時不先常數摺疊。**教材 ac 的右結合差異見 p.4。**

F4｜給 NFA，轉 DFA

完整演算 p.8

① 起始 A=close({q_0})。② 對 A 的每種 c：聯集 move(A,c)，再 close。③ 新集合命名並入 worklist；相同集合沿用名稱。④ 重複直到 worklist 空。⑤ 含任一 NFA 接受 state 的集合標接受；補 ∅ sink。

$$\delta_D(S,c)=\text{close}(\text{move}(S,c))$$

$$S \text{ 接受} \Leftrightarrow S\cap F_N\neq\varnothing$$

微型題（自編）：Q={1,2,3}、Σ={a}，start=1、F={3}；只有 1—λ→2、2—a→3，其餘無邊。

A=close({1})={1,2}；move(A,a)={3}，再 close 得 B={3}；B 讀 a 得 X=∅。

DFA / NFA 集合	a	接受？
→A={1,2}	B	否
★B={3}	X	是
X=∅	X	否

a ✓：A→B。**λ ✗：**停 A。**aa ✗：**A→B→X。判整串要讀完；NFA 是**存在**成功路徑。Move on one symbol, take epsilon closure, and reuse equal subsets.

閉包可走零條 λ 邊，包含起點；∅ 若不可達就不列進可達集合。最小化：先分接受 / 拒絕，再按後繼組別反覆拆；補充例 p.9。

F6｜判錯誤 / 解釋 Why CFG

詳 p.3

先逐層判：字元能切 token？→ token 符合 grammar？→ 宣告 / 型別符合規則？判定需依題目所給語言。

C-like 假設例	第一個問題
@ 無合法 token pattern	Lexical
x = ; 缺 RHS	Syntax
x 未宣告但 x=1 文法合法	Static semantic

The input violates the token pattern / grammar / declaration or type rules. 作答要補**具體原因**，不要只寫 error 名稱。

> **Why CFG：**任意深度巢狀 → RE / 有限狀態不夠 → CFG 可遞迴描述 → 實用 parser；宣告與型別交 semantic analysis。CFGs describe recursive, nested syntax beyond regular languages and support practical parsing.

交卷前 10 秒：token type≠lexeme；λ≠∅；字元≠token；NFA state≠DFA 子集；closure 不吃字元；接受完整字串要讀完；AST 的左右 / 敘述順序要保留。**看不懂題目名詞先查 p.1，數學構造直接照 F3 / F4。**

Chapter 3 | Scanning 考場速查

找答案：名詞 → p.1；流程 → p.2；CFG → p.3；AST → p.4；RE → p.5；掃描 → p.6；RE→NFA → p.7；子集構造 → p.8；abb / 最小化 → p.9；綜合題 → p.10。
題型依據：第一次：Explain、Difference、情境判斷；第二次：Why CFG、短程式 AST。以下是**複習用作答範本與自編練習**，不代表本次考題或教師標準答案。

1 | Why use a context-free grammar instead of others?

Quiz 2；Ch3 PPT 51–52＋補充

可直接寫：Context-free grammars can describe recursive and nested syntax, such as balanced parentheses, which regular grammars cannot express in general. They offer a useful balance between expressive power and practical parsing. More powerful grammar classes make general parsing harder, while declaration and type constraints are usually checked separately during semantic analysis.

中文抓分：① 足以表達無上限巢狀 / 遞迴結構；② 比 regular grammar / RE 強；③ 比更強文法容易設計實用 parser；④ 語法與型別 / 宣告檢查分工。**不要只寫「CFG 比較快」**，RE 的 DFA 掃描通常反而很有效率。

規格	擅長什麼	作答時的界限
RE / regular grammar	identifier、number、keyword 等 token 形狀	有限狀態；不能一般性地配對任意深度括號
CFG	expression、statement、program 的遞迴結構	A → α：左側只有一個 nonterminal；不保證無歧義或 LL(1)
Context-sensitive / unrestricted grammar	可描述更廣的字串限制	一般剖析更複雜；日常 compiler 多將相關限制交 semantic analysis

最短版：CFGs express nested program structure beyond regular languages and support practical parsing; semantic analysis handles declaration and type constraints.

2 | Scanner / Parser / Semantics

階段	輸入 → 輸出 / 任務
Scanner / lexer	characters → tokens；RE 辨認詞素
Parser	tokens → syntax structure / AST；依 CFG
Semantic analysis	AST ＋ symbol table；宣告 / 型別檢查
Code generation	checked representation → target code

The scanner groups characters into tokens. The parser checks token structure using a CFG and builds a syntax representation. Semantic analysis checks meaning-related constraints such as declarations and types.

建 scanner：RE → NFA → DFA → table / code。
跑 scanner：source characters → tokens。
兩條流程不可倒混；scanner **不負責畫 AST**。

3 | Token / Lexeme / Attribute

PPT 3、31

A token has a type and, when needed, an attribute. The lexeme is the actual source character sequence matched by the token pattern.

`count`：type=ID，lexeme="count"，attribute 可為符號表索引。
`007`：lexeme="007"，數值 attribute 可為 7。

Pattern 是規格；**lexeme** 是實際字串；**token type** 是 parser 用來分類的名稱。不同 lexemes 可屬同一 type。

4 | CFG 強在哪？一個可寫的例子

令 L = { $a^n b^n$ | n ≥ 0 }，CFG：

$$S \to aSb \mid \lambda$$

`S ⇒ aSb ⇒ aaSbb ⇒ aaaSbbb ⇒ aaabbb`。每次多一個 a，同時多一個 b；λ 結束，a 都在 b 之前。

有限 DFA 反證：若有 m 個狀態，讀 a^0、a^1、…、a^m 後至少兩個字首 a^i、a^j（i<j）到同一狀態。再接 b^i，DFA 必須同判；但 $a^i b^i$ 應收、$a^j b^i$ 應拒，矛盾。

> **不要寫：**「aaabbb 不是 regular」。單一字串可直接用 RE `aaabbb`。不能表達的是**任意 n 的整個等量語言**。

括號例：S → (S)S | λ 可表達平衡括號。CFG 的「context-free」指改寫 A 不需看左右上下文，**不是程式不用考慮 scope**。

5 | 情境題：先說哪一層失敗

現象（依指定語言規則）	階段
@ 無任何 token pattern	Lexical error
x = ;，每個 token 合法但缺 RHS	Syntax error
x 未宣告，但 x=1 文法合法	Static semantic error
文法 / 型別合法，結果違反人的意圖	可能是 logic error

A syntax error violates the grammar; a static semantic error violates declaration or type rules despite a grammatically valid token sequence.

AST｜看到短程式，怎麼畫？

6｜四步畫樹＋一個驗算

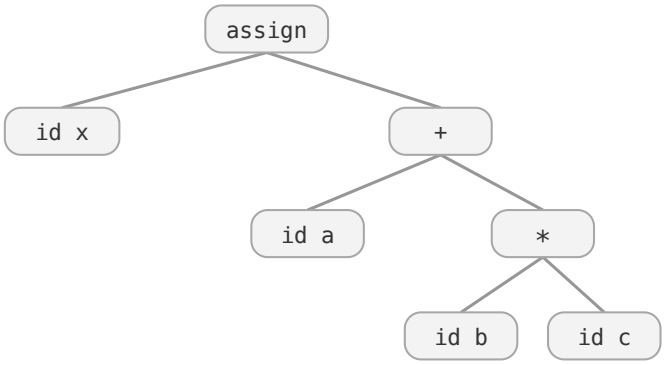
Ch2 PPT 27–28、39–42＋自編例題

① **分敘述：**program / block 依來源順序接 declaration、assignment、print 等。② **找最外層：**如 `x=E` 先畫 assign，左目標、右 RHS。③ **遞迴分解：**先按括號、再優先序、再結合性決定 expression 的根。④ **保留：**operators、識別字、常數、必要的敘述結構；省略純語法包裝與分隔符。⑤ **驗算：**由樹還原括號式，檢查左右、先後與題目相符。

An AST represents the essential syntactic structure using operators and operands. It omits grammar-only nodes and punctuation while preserving precedence, associativity, and statement order.

> **第二次小考未附實際短程式。**下方 A/B 是**一般 C-like 補充練習**，不是原小考答案；教材 ac 沒有乘法、括號或分號。先辨認題目採哪套語言。

7｜A：x = a + b * c;

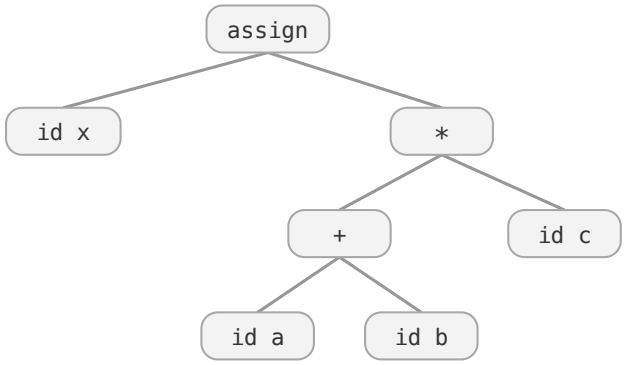


RHS 的根是 +：因 * 優先於 +，`b*c` 作一個子樹。整句的根仍是 assign。

```
assign(id(x),
       plus(id(a), times(id(b), id(c))))
```

上式是樹的文字記法；要求畫樹時，畫節點與邊。葉標 a 或 id(a) 須一致。

8｜B：x = (a + b) * c;



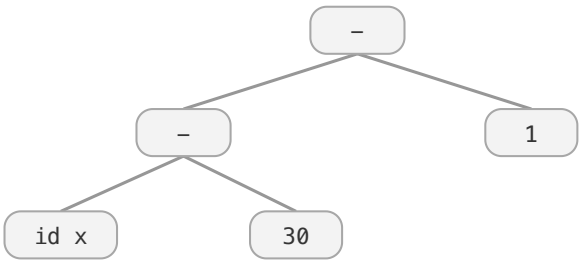
RHS 的根改成 *：括號強制先加；AST 通常不畫「(」「)」節點，**樹形本身保留分組**。

```
assign(id(x),
       times(plus(id(a), id(b)), id(c)))
```

Expr、Term、Factor、分號、λ 屬 parse tree 的語法細節。

9｜結合性：同優先序時怎麼拆？

一般 C 減法左結合：`x-30-1` → `(x-30)-1`；最外層是**最後一個 −**。



x=100 時左樹得 69；右樹 `x-(30-1)` 得 71。左右運算元不能交換。

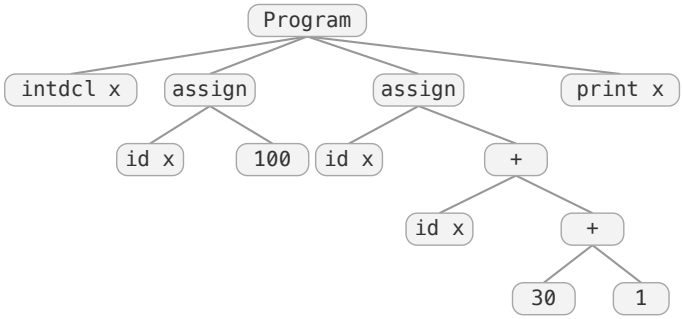
一般 C 指定右結合：`x=y=z` → `assign(x,assign(y,z))`。**一元負號** `-x` 畫 neg(x)，與 binary minus(L,R) 分開。

右遞迴文法不必強迫右結合 AST；semantic actions 可累積左子樹。以題目指定的語意 / 教材圖為準。

10｜教材 ac：原圖是右結合

Ch2 p.42

`i x  x=100  x=x+30+1  p x`



重繪 Ch2 p.42 的樹形；補明 declaration / token 名稱。

> **依教材圖：**plus(x,plus(30,1))。**一般左結合：**plus(plus(x,30),1)。本例整數值同為 131，但不能任意把右樹當所有語言的標準。

11｜容易失分：AST ≠ Parse Tree ≠ Decorated AST

表示	保留什麼	作答提醒
Parse tree	文法的 nonterminals、terminals、每次改寫	葉序要符合 token 序列；依要求畫 λ
AST	核心操作與 operands、敘述 / 宣告結構	保留所有敘述；不要只畫 RHS 或算出結果
Decorated AST	AST 加型別、符號資訊，可能插 int2float	只有要求型別分析時才加入；普通 AST 不先常數摺疊

Why use an AST? It removes grammar-specific detail and gives later analysis and code generation a compact representation of program structure.

Regular Expressions | 寫規格與判合法

12 | RE 是語言：一個字串集合

A regular expression specifies a regular language: the set of strings that match its pattern. A finite automaton recognizes that language.

Σ：有限字母表；**Σ***：Σ 上所有有限長字串。**λ / ε**：空字串，|λ|=0。**∅**：空集合，沒有任何字串。**L(λ)={λ} ≠ L(∅)=∅**；教材用 0 表空集合，不是 literal "0"。

形式	語意 / 展開	例與陷阱
A | B	L(A) ∪ L(B)；選任一支	a|bc 接受 a 或 bc，不接受 abc
AB	先 A 再 B；{xy | x∈L(A), y∈L(B)}	一般 AB ≠ BA；Aλ=λA=A
A*	0 次以上；λ | A | AA | …	一定含 λ；∅*={λ}
A+	1 次以上；AA*	若 A 自己含 λ，A+ 仍可能含 λ
A?	0 或 1 次；A | λ	(ab)? 接受 λ 或 ab
A^k	串接恰 k 次；A^0=λ	$(ab)^2$ 是 abab
Not(C)	字元集合 Σ\C，C⊆Σ	只排除單一字元集合；非任意語言補集

優先序：分組 (…) → postfix * + ? → 串接 → |。**Literal**：要匹配字面 *、+、.，依工具 syntax 跳脫或用字元類別。`[.]` 是字面點；多數 regex 工具的 `.` 是 wildcard，是否含換行取決於 dialect / 設定。

13 | 最常見的分組判斷

RE	整串可接受	整串不可接受
ab*	a, ab, abb, …	λ, b, abab
(ab)*	λ, ab, abab, …	a, abb
(abc+)+	abc, abcc, abcabc	λ, ab, abbc
(a|b)*abb	abb, aabb, babb, ababb	λ, ab, abba

不要看字元各自出現過就說合法。`(abc+)+` 每塊都是一個 a、一個 b、至少一個 c，然後整塊至少一次。

`A∅=∅A=∅`；`A|∅=A`。空字串是串接的單位元，空集合是會讓串接全空的集合。

14 | Identifier：本章特有規格 p.15、32

```
L = [A-Za-z]    D = [0-9]
L(L|D)*(_(L|D)+)*
```

讀法：字母開頭；後接任意字母 / 數字；每個底線後至少一個字母 / 數字。允許 `foo_2_bar`，拒 `_foo`、`foo_`、`foo__bar`。

底線不是任意可放；`[A-Za-z_][A-Za-z0-9_]*` 是**另一套**常見 ID 規格，不能代替本頁教材。

15 | 格式符合不代表語意正確

p.16 日期規格允許年 1900–2099、月 01–12、日 01–31，所以 `2001-02-31` 形狀符合，日曆不合法。兩個分隔符獨立，`2001/02-03` 也符合。

字串 `"[^"\r\n]*"` 禁 quote / CR / LF，但**沒定義跳脫引號**；不能當完整 C string 規格。

16 | 科學記號：兩層 optional HW 3；補充修正版

```
^[0-9]+([.][0-9]+)?
        (e[+-]?[0-9]+)?$
```

排版換行僅供閱讀；實際 pattern 不含換行或上述空格。

① 整個 exponent 可省略：`(e…)?`。**② exponent 內符號可省略：**`[+-]?`。e 後數字始終**至少一位**。沿用 ac：小數點兩側必須有 digit。

測試字串	整串結果 / 理由
12, 3.2	✓ exponent 可省
1.0e10, 123e-22, 12e+3	✓ 無號 / 負號 / 正號
12e, 1.2e-	✗ 缺 exponent digits
.1, 10., 3.e2	✗ 小數兩側須 digit
1E2	✗ 這個規格只收小寫 e

p.54 原解答的缺漏：e 部分變必填；只寫 -? 漏正號；fnum 小數前少 +。問「依題意擴充」使用上方修正版；NUM 如何分 int / float 是另外的語意政策。

Make the entire exponent optional, allow an optional plus or minus sign, and require at least one exponent digit.

17 | Whole match / Search / Tokenize

整串驗證：要求所有字元屬同一 pattern，例 `^…$`（依工具模式）。**Search**：找一段子字串。**Scanner**：從目前位置取合法 token 前綴，然後繼續。

`12e` **不符合完整 NUM**；但某 scanner 可切 `NUM(12), ID(e)`。要把它報 malformed number，須另定規則。不要由整串 regex 拒絕直接斷言 lexer 必報錯。

Dialect 提醒：理論 RE 的 λ、Not 是數學記法；工具的 `\d`、dot、anchors、BRE / ERE 有自己的規則。上方數值採 POSIX ERE 形式；不用把所有語法混在同一答案。

Finite Automata × Scanner 行為

18｜五元組與 DFA / NFA

PPT 23–36

$$M = (Q, \Sigma, \delta, q_0, F)$$

Q：有限 states；**Σ**：輸入字母表；**δ**：轉移；q_0：起始 state；**F⊆Q**：接受 states。圖中 incoming arrow 是起始，雙圈是接受；一個 state 可同時是兩者。

比較	DFA	NFA（本章允許 λ 邊）
轉移	δ: Q×Σ → Q；每個 state / 字元唯一後繼	Δ: Q×(Σ∪{λ}) → $\mathcal{P}$(Q)；後繼是一個集合
λ / ε 邊	無；每一步吃一個字元	可有；移動不吃字元
接受完整 w	讀完所有字元後，當前 state 在 F	存在一條吃完整 w、可含 λ 的路徑到 F
表達能力	Regular languages	同 DFA；較方便組合，並沒有更強

A DFA has one next state for each state and input symbol. An NFA may have multiple choices and epsilon transitions. They recognize the same class of languages.

> NFA 不是要求所有路徑成功；一條成功即可。DFA 中途到接受 state 也不能忽略後面的字元。**∅ 集合**可成為 DFA 的 dead state，與空字串 λ 不同。

19｜Comment DFA：逐字追蹤

p.25–30

教材：`//(Not(EOL))*EOL`。OTHER 是非 /、非 EOL 字元；X 表拒絕 sink。EOF 是結束條件，**不是輸入字元 EOL**。

State	/	EOL	OTHER	結束時
→1	2	X	X	拒
2	3	X	X	拒
3	3	4	3	拒
★4	X	X	X	接受
X	X	X	X	拒

`//hi\n`：1→2→3→3→3→4 ✓。`//hi` 在 EOF 留在 3 ✗，因教材要求 EOL。這不是宣稱所有真實語言都拒絕檔尾註解。

Table-driven：資料表 δ + 通用 driver。**Explicit control**：if / switch / loops 直接編入轉移。**Transducer**：辨識之外還產出 token / attribute，例如 digit 串轉數值。

Table-driven scanning separates transition data from control code. Explicit control embeds transitions in branches and loops.

20｜整串辨識器手算模板

設 q=q_0 → 依每個字元查下一 state → 不合法邊到 X → **字串讀完**才判 q∈F。表中沒畫的邊通常表示失敗；若要求完整 DFA，就補 sink。

字母表先寫清楚：本頁 comment 表用「字元類別」合併大量字元；小題 a/b/c 的表則按個別符號列。

21｜Maximal munch + Tie priority

p.20；flex 補充

Choose the longest matching lexeme. If multiple flex rules match the same length, choose the rule listed first.

keyword 與 ID 重疊：把 keyword rule 放 ID 前面。同長才比順序；**不能先看到 keyword 就停止**。

輸入	應選	理由
float	FLOAT keyword	與 ID 同長；keyword 規則先列
floatx	ID(floatx)	ID 6 字元長於 keyword 5
int7	ID(int7)	以本章多字元 ID 規格為例
5+10	NUM(5), PLUS, NUM(10)	每次從目前位置重新匹配

通用 scanner 需記最後接受：掃描時更新 `last_accept_state` 與 `last_accept_position`；遇到死路，回到最後接受處，回傳那個 token，剩下字元交下一輪。

> **反例**：只有 `a`、`abc` 兩種 token，輸入 `abx`。讀 a 已接受；讀 b 尚有可能；讀 x 失敗。應回退輸出 token `a`，從 b 繼續 / 報錯。**只檢查最後 state 會漏掉合法前綴**。

教材 p.28 的簡化 driver 只查停下時的 state，不能直接視為處理所有 token 規格的 maximal-munch implementation。

22｜小 token 題：honeycone=5+10*num;

```
ID(honeycone), ASSIGN(=), NUM(5),
PLUS(+), NUM(10), TIMES(*), ID(num),
SEMICOLON(;)
```

共 8 tokens，與空白 / 字元個數不同。此例取自 p.3 的 C-like token 示意，**不是** ac 的單字母 identifier 文法。

略過 whitespace / comment 的 action 可繼續掃，不交 token 給 parser。`yylval` 傳必要 attribute；`return` 交 token type。

RE → NFA｜按結構拼，別猜箭頭

23｜先解析 RE，再組片段

PPT 37–41；補充操作順序

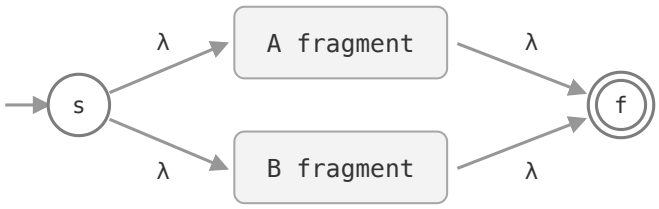
順序：① 先圈群組與 operators；② literal 用一條吃字元的邊；③ 按 |、串接、* 組合；④ 標一個整體 start 與 accept；⑤ 用 λ、最短合法字串、近似錯例驗算。狀態編號可不同，**結構與語言正確才重要**。

a：s —a→ f；**λ：**s —λ→ f；**∅：**沒有從 s 到 f 的路徑。所有子片段 state 先重新編號以免撞名；組合後移除子片段的「整體接受」標記，只保留完整表達式的出口。

24｜三個可套模板

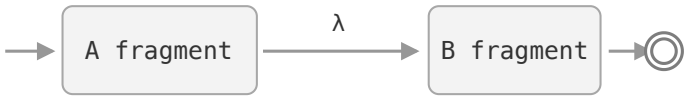
依教材重繪

A | B：分支取任一條



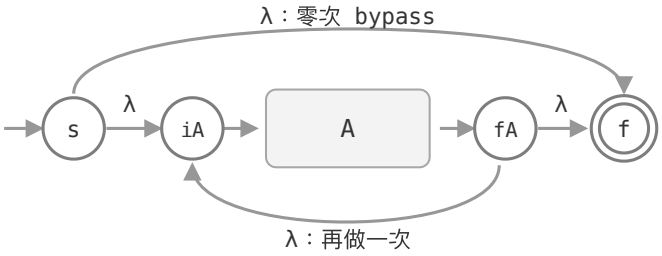
新起點 λ 分到 A / B；兩出口 λ 到新終點。只要一支完成即可接受。

AB：先 A，再 B



A 的出口 λ 接 B 的入口；**只讀完 A 不算完成 AB**。

A*：零次旁路＋重複迴圈



A+：保留重複邊，移除新 s→新 f 的零次旁路。**A?：**A | λ；有旁路，沒有重複邊。

> 若 A 自己可接受 λ，去掉外部 bypass 後 A+ 仍可能接受 λ。例 (a*)+ 可收空字串；「+ 不收 λ」須先確認內層不收 λ。

25｜HW 5(d)：(bc)*d

PPT 55–57

語言：零個以上 bc，最後**恰一個 d**。λ 不收，但 d 收。教材 NFA：start=1，F={6}，Σ={b,c,d}。

NFA	b	c	d	λ
→1	∅	∅	∅	{2,5}
2	{3}	∅	∅	∅
3	∅	{4}	∅	∅
4	∅	∅	∅	{2,5}
5	∅	∅	{6}	∅
★6	∅	∅	∅	∅

分解：1→5 的 λ 允許零次 bc；2—b→3—c→4，4→2 的 λ 再做一次；4→5 的 λ 離開迴圈，再吃 d 到 6。

轉 DFA：先算集合（公式見 p.8）

A={1,2,5}；B={3}；C={2,4,5}；D={6}；X=∅。→ 是 start，★ 是接受。

DFA	b	c	d
→A	B	X	D
B	X	C	X
C	B	X	D
★D	X	X	X
X	X	X	X

一格怎麼算：A 在 b 下只有 2→3，所以 move={3}、close={3}=B。B 在 c 下到 4，再走 λ 到 2、5，得到 C。

bcd：A→B→C→D ✓。**bc：**A→B→C ✗。**bcdd：**A→B→C→D→X ✗；先到 D 不代表剩下 d 可忽略。

最小化補充：A、C 轉移完全相同且都拒絕，可合併；連同 B、D、X 得 4 states。先做完子集構造，再談合併。

26｜考試畫 NFA 的五個檢查點

Start / F / Σ 都有；每條邊明寫字元或 λ；* 的零次 / 重複都有；+ 不誤加外部零次邊；串接不讓前半完成就接受。

Construction answer: Build NFAs for the subexpressions, connect them according to alternation, concatenation, and closure, and mark only the final exit of the complete expression as accepting.

NFA → DFA｜集合就是一個狀態

27｜三個公式＋接受條件

PPT 42–48

$$S_0 = close(\{q_0\})$$
$$move(S,c) = \cup_{s\in S}\ \Delta(s,c)$$
$$\delta_D(S,c) = close(move(S,c))$$
$$S \text{ 是接受 state} \Leftrightarrow S \cap F_N \neq \varnothing$$

close(S) / λ-closure：從 S 本身開始，沿**零條或多條 λ 邊**到的全部 states。遇循環就去重，反覆加入到沒有新 state；不吃字元。
move(S,c)：所有成員吃**一個 c**的後繼聯集；之後一定再 close。

Each DFA state represents all possible NFA states after the same input prefix, including epsilon reachability. It is accepting if it contains at least one accepting NFA state.

28｜教材 Fig 3.24 的原 NFA

p.43

start=1，F={5}，Σ={a,b}；只有 1→2 是 λ。

NFA	a	b	λ
→1	{3}	{1}	{2}
2	{4,5}	∅	∅
3	∅	{4}	∅
4	{5}	{5}	∅
★5	∅	∅	∅

Step 1：起始閉包

{1} → 加入 2 → {1,2}，再無新 λ 後繼。令 A={1,2}。不含 5，故**空字串拒絕**。

> **p.47 原句把 3 寫入 closure，應校正。**1→3 是 a 邊；不讀 a 就不能把 3 加進起始閉包。

Step 2：算 A 的所有字元

```
move(A,a) = {3} ∪ {4,5}
          = {3,4,5}
close(...) = {3,4,5} = B

move(A,b) = {1} ∪ ∅ = {1}
close({1}) = {1,2} = A
```

B 含 5，標接受；A 已存在，不再造新 state。**同一集合就是同一 state**，排列順序不影響身分。

Step 3：算 B、其餘新集合

B 在 a 下只有 4→5，得 D={5}；在 b 下由 3→4、4→5，得 C={4,5}。C 讀 a / b 都得 D。D 讀任何字元皆 ∅，令 X=∅。

29｜完整 DFA：每一格都交代

A={1,2}；B={3,4,5}；C={4,5}；D={5}；X=∅。B、C、D 接受，因都含 5。

DFA	a	b
→A	B	A
★B	D	C
★C	D	D
★D	X	X
X	X	X

Worklist：a 先、b 後，FIFO

取出	本輪新增	本輪後 queue
A	B	[B]
B	D, C	[D,C]
D	X	[C,X]
C	無	[X]
X	無	[]：結束

seen 存已見集合；**worklist** 存尚未處理全部字元的集合。每個新集合只加入一次；表要包含已見集合的轉移。

驗算幾個字串

輸入	走 DFA	最後
λ	A	拒
bbb	A→A→A→A	拒
a	A→B	接受
ab	A→B→C	接受
abb	A→B→C→D	接受
aaa	A→B→D→X	拒

> **∅ 不是「未算」**：沒有 NFA 路徑存活，成為 DFA 的拒絕 sink；在所有字元下仍到 ∅。原圖若省略 sink 是部分圖，完整轉移表要補上。

30｜Why correct? Why terminate?

英文說理題＋補充不變量

Initially, epsilon closure gives every NFA state reachable without input. After each symbol, move collects all symbol transitions and closure adds every epsilon–reachable state. Thus the subset always equals the possible NFA positions after that prefix. Both automata accept exactly when this subset intersects the NFA accepting set.

終止：n 個 NFA states 至多有 2^n 個子集；只處理可達集合，且每個至多一次。2^n 是**上限**，不是每題一定生成的個數。此例 5 個 NFA state → 5 個可達 DFA state。

(a|b)*abb｜構造、追蹤與合併

31｜先用一句話說出語言，再畫狀態機

Over the alphabet {a,b}, (a|b)*abb denotes all strings ending in abb.

前綴可為任意 a / b 字串（含 λ），尾端必須是 abb；最短接受字串是 **abb**，不是 ab。教材 p.41 另有 9-state NFA，**本頁只用 p.49 的編號**。

32｜原 NFA：q_0=0、F={10}

NFA	a	b	λ
→0	∅	∅	{1,7}
1	∅	∅	{2,4}
2	{3}	∅	∅
3	∅	∅	{6}
4	∅	{5}	∅
5	∅	∅	{6}
6	∅	∅	{1,7}
7	{8}	∅	∅
8	∅	{9}	∅
9	∅	{10}	∅
★10	∅	∅	∅

結構：0→7 允許零次前綴；1→2 / 4 選 a / b；3 / 5→6→1 重複；6→7 離開迴圈。7—a→8—b→9—b→10 完成尾碼。

起始閉包 → 第一個 a

```
close({0}) = {0,1,2,4,7} = A
move(A,a)  = {3,8}
close({3,8}) = {1,2,3,4,6,7,8} = B
```

3 的 λ 到 6，再到 1、7，再到 2、4。**閉包不是只走一次 λ**；要一路到固定點。

abb 的完整集合追蹤

A —a→ B —b→ D —b→ E。最後含 10，接受。NFA 的「任意前綴」與「尾碼開始」可同時存活，DFA 用集合一起保留。

33｜所有可達子集與完整 DFA

名稱	NFA state 集合
→A	{0,1,2,4,7}
B	{1,2,3,4,6,7,8}
C	{1,2,4,5,6,7}
D	{1,2,4,5,6,7,9}
★E	{1,2,4,5,6,7,10}

DFA	a	b
→A	B	C
B	B	D
C	B	C
D	B	E
★E	B	C

逐輪：A 發現 B、C；B 發現 D；C 無新集合；D 發現 E；E 無新集合。只有 E 含 10。在 Σ={a,b} 下**沒有可達 ∅**，不可硬加進可達集合清單。

字串	DFA path	結果
aabb	A→B→B→D→E	✓
babb	A→C→B→D→E	✓
abba	A→B→D→E→B	✗
ababb	A→B→D→B→D→E	✓

E 是接受、不是停止：追加 a 會回 B，不再接受；追加 b 回 C。問整串辨識，必須把輸入讀完。

34｜最小化：先接受 / 拒絕，再拆　補充

兩 states 只有對**所有後續字串**接受結果一致才可合併；不是「都拒絕」就能合併。先按 F 分組，再按各字元後繼的組別反覆細分。

輪	分組（每格表示一組）
P_0	{E} / {A,B,C,D}
P_1	{E} / {D} / {A,B,C}
P_2	{E} / {D} / {B} / {A,C}
P_3	與 P_2 相同；固定點

D 的 b 後繼接受，所以先分 D；接著 B 的 b 後繼到 D，所以分 B。A、C 兩個字元的組別始終相同，可合併。

35｜最小 DFA：用尾碼進度理解四個 states　補充，不是原投影片新增要求

最小 state	a	b	已讀字首的最長尾段 / 接受
→U={A,C}	B	U	λ：尚未匹配 abb 的非空字首
B	B	D	a
D	B	E	ab
★E	B	U	abb；接受

不能再合併：E 與其餘由 λ 區分；D 與 B / U 由 b 區分；B 與 U 由 bb 區分。這些 distinguishing suffixes 會讓其中一方接受、另一方拒絕，故至少需要四個 states。

綜合題 × 英文短答 × 最後檢查

36｜(a|(bc)*d)+：先理解「一塊」

PPT 58–60

一塊是 a，或零個以上 bc 加 d；整串由**至少一塊**串接。因此 d、a、ad、bcd、abcbcdad 接受；λ、bc、bd 拒絕。**內層 * 可零次，不代表外層 + 可零塊。**

37｜教材 NFA：start=1，F={12}

Σ={a,b,c,d}；表中 — 表示 ∅（無後繼）。

state	a	b	c	d	λ
→1	—	—	—	—	{2}
2	—	—	—	—	{3,6}
3	{4}	—	—	—	—
4	—	—	—	—	{5}
5	—	—	—	—	{2,12}
6	—	—	—	—	{7,10}
7	—	{8}	—	—	—
8	—	—	{9}	—	—
9	—	—	—	—	{7,10}
10	—	—	—	{11}	—
11	—	—	—	—	{5}
★12	—	—	—	—	—

抓結構：2 分到 a 或 (bc)*d；完成一塊到 5；5→2 可再做一塊、5→12 可結束。沒有起始→12 的零次 λ 旁路。

38｜英文短答：Definition / Difference

What is epsilon closure?

It is the set of states reachable from a given set through zero or more epsilon transitions, including the original states.

Is an NFA more powerful than a DFA?

No. Both recognize regular languages. Subset construction converts an NFA into an equivalent DFA.

What is a dead state?

It is a non-accepting state from which no accepting state is reachable. The empty subset in subset construction is a sink that loops on every input symbol.

39｜DFA 集合與完整轉移表

state	NFA states
→A	{1,2,3,6,7,10}
★B	{2,3,4,5,6,7,10,12}
C	{8}
★D	{2,3,5,6,7,10,11,12}
E	{7,9,10}
X	∅

DFA	a	b	c	d
→A	B	C	X	D
★B	B	C	X	D
C	X	X	E	X
★D	B	C	X	D
E	X	C	X	D
X	X	X	X	X

逐輪（a,b,c,d；FIFO）：A 新增 B、C、X、D；B 無新；C 新增 E；X、D、E 無新。含 12 的 B / D 才是接受。

> abcbcdad：
> A → B → C → E → C → E → D → B → D ✓

最短反例：bc 停 E，尚缺 d；bd 由 C 在 d 下到 X，少了 c。將路徑與「一塊」的定義互相核對。

40｜英文短答：Why / Scenario

Why convert RE to NFA and then to DFA?

NFAs are easy to build compositionally from regular expressions. A DFA then supports deterministic scanning with one next state per input symbol.

When is a subset accepting?

When it contains at least one accepting NFA state; it does not require all member states to be accepting.

Why must a scanner remember the last acceptance?

A longer candidate may fail after a shorter valid token prefix. Remembering the last accepting position allows the scanner to return the longest valid prefix.

41｜考場作答模板：把過程寫成能給分的形式

> **Explain：**定義 → 工作 / 輸入輸出 → 小例子。**Why：**能力需求 → 比較限制 → 實作理由。**Draw AST：**寫語言假設 → 根與子樹 → 優先序 / 結合性。**Convert FA：**列 Q、Σ、start、F → close 起始 → 每字元 move 再 close → 去重 → 完整表 / 圖 → 標接受 → 測 λ、最短成功、近似失敗。

最常漏：λ≠∅；DFA 接受要讀完；NFA 接受是「存在」；closure 走到底；* 含零次；+ 看內層是否含 λ；keyword 同長才比規則；AST 不先算成常數；依題目採教材或一般語意。

來源 / 範圍：Ch03 scanning.pptx（60 slides）與六份伴讀；前兩次小考題型；AST 課本例取 Ch2 p.42。圖均重繪；C-like 練習、maximal munch、反例、最小化為補充。10 頁 A4（雙面 5 張），未假定允許頁數上限。
教材疑點已辨析：Ch3 p.47 閉包、p.51 字串 vs 語言、p.54 exponent；Ch2 p.42 右結合。Ch2 p.34 / 43 待核對仍見舊筆記。
參照：原 PPTX · 本章導讀 · Quiz 1 · Quiz 2 · flex Matching。
列印：A4 / 100% / 關閉瀏覽器自帶頁首頁尾；HTML 離線可讀。沒時間時：p.1 查名詞、p.2 按流程；CFG / AST 詳解 p.3–4；p.9 最小化為補充。